ACTIVIDAD 3 - PROYECTO DE MONITORIZACIÓN

Cristóbal Suárez Abad

OPTATIVA - 2º ASIR

Índice

Paso 1: Configuración del Servidor de Logs (Rsyslog) en 10.2.7.99 3
Paso 2: Configuración de los Emisores (Logs y Métricas) 5
1. Router Mikrotik (10.2.7.3) 5
2, 3 y 4. Servidor FOG (10.2.7.5) (Linux, Apache y MariaDB) 6
Recolectores de Métricas 7
Preparación en la VM de FOG (10.2.7.5) 7
Configurar el Exportador de MariaDB (MySQL) 8
Paso A: Crear el usuario en MariaDB 8
Paso B: Descargar e instalar el binario 8
Paso C: Crear el servicio en el sistema 9
Paso D: Iniciar el exportador 10
Paso E: Credenciales 10
Configurar el Exportador de Apache 12
Paso A: Activar mod_status en Apache 12
Paso B: Descargar e instalar el binario 12
Paso C: Crear el servicio en el sistema 12
Paso D: Iniciar el exportador 13
Comprobación final de puertos 14
5. Cliente Ubuntu 15
Paso 1: Logs del Sistema (Syslog) 15
1.Editar la configuración de Rsyslog:En el cliente Ubuntu. 15
2.Añadir la regla de reenvío:Al final del archivo. 15
3.Reiniciar el servicio:Aplicar cambios. 15
Paso 2: Métricas de Hardware (Node Exporter) 16
Paso 3: Despliegue de la Monitorización en 10.2.7.99 (Docker Compose) 17
Configuración del router Mikrotik 17
docker-compose.yml 18
prometheus.yml 20

promtail-config.yml 21
Regla de firewall 22
Pasos finales recomendados en la máquina para ponerlo en marcha: 22
Consideraciones 23
Comprobación de los LOGS 24
Paso 4: Grafana. Creación de Connections, Dashboards y Alertas 27
Añadir “Data Source” 27
Dasboards 30
Para Equipos Linux: 30
Para router Mikrotik: 34
Dashboard personalizado 36
Servidor FOG: 36
Cliente Linux: 38
Router Mikrotik: 40
LOGS 41
Servicios 43
Apache 43
MariaDB 44
Servicios de FOG 44
Alertas de Eventos Críticos 46
Paso 1: Crear el Bot de Telegram 46
Paso 2: Configurar el Punto de Contacto en Grafana 50
Paso 3: Creación de Alertas 54
Paso 4: Creación de alerta para el uso de CPU del servidor 62
Prueba de la regla: 64

Paso 1: Configuración del Servidor de Logs (Rsyslog) en 10.2.7.99

Configuraremos el servicio rsyslog nativo de la máquina 10.2.7.99. Edita el archivo /etc/rsyslog.conf:

```
sudo nano /etc/rsyslog.conf
```

Añade o desbloquea las siguientes líneas para activar la escucha por UDP y TCP en el puerto 514, y aplica las reglas de filtrado solicitadas:

```
# Activar módulos de escucha UDP y TCP
module(load="imudp")
input(type="imudp" port="514")

module(load="imtcp")
input(type="imtcp" port="514")

# 1. FILTRO DE ALERTAS TOTALES (Aquí interceptamos TODO lo grave de todos los equipos)
# Se ha añadido opciones para pillar los tipo Test y también para los mensajes de Mikrotik, que son diferentes
# El syslogseverity 3 pilla los crit, alert y emerg
if ($syslogseverity <= 3 or $msg contains "level=ERROR" or $msg contains "GRAFANA_TEST" or $programname contains "critical" or $msg contains "crit>
  action(type="omfile" file="/var/log/ALERTA_TOTAL.log")
  stop
}


# 2. Segregación de Base de Datos
if $programname == 'mysqld' or $msg contains 'mariadb' or $msg contains 'mysql' then {
  action(type="omfile" file="/var/log/BBDD_LOG.log")
  stop
}

# 3. Todo lo demás que sea remoto va a las carpetas dinámicas
if $fromhost-ip != '127.0.0.1' then {
  $template DynamicFiles,"/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log"
  *.* ?DynamicFiles
  stop
}
```

```
# Esta parte ya venía configurada
## Activar módulos de escucha UDP y TCP
# provides UDP syslog reception
module(load="imudp")
input(type="imudp" port="514")

# Esto hubo que descomentarlo
# provides TCP syslog reception
module(load="imtcp")
input(type="imtcp" port="514")
```

```
# 1. FILTRO DE ALERTAS TOTALES (Aquí interceptamos TODO lo grave de todos los equipos)
# Se ha añadido opciones para pillar los tipo Test y también para los mensajes de Mikrotik, que son diferentes
# El syslogseverity 3 pilla los crit, alert y emerg
if ($syslogseverity <= 3 or $msg contains "level=ERROR" or $msg contains "GRAFANA_TEST" or $programname contains "critical" or $msg contains "crit
    action(type="omfile" file="/var/log/ALERTA_TOTAL.log")
    stop
}


# 2. Segregación de Base de Datos
if $programname == 'mysqld' or $msg contains 'mariadb' or $msg contains 'mysql' then {
    action(type="omfile" file="/var/log/BBDD_LOG.log")
    stop
}

# 3. Todo lo demás que sea remoto va a las carpetas dinámicas
if $fromhost-ip != '127.0.0.1' then {
    $template DynamicFiles,"/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log"
    *.* ?DynamicFiles
    stop
}
```

Reinicia el servicio:

sudo systemctl restart rsyslog

Para el archivo de Alertas debes darle los permisos necesarios.

```
sudo chown syslog:adm /var/log/ALERTA_TOTAL.log

sudo chmod 664 /var/log/ALERTA_TOTAL.log
```

Paso 2: Configuración de los Emisores (Logs y Métricas)

1. Router Mikrotik (10.2.7.3)

- **Logs (Syslog):** Accede por WinBox o SSH y ejecuta:

```
/system logging action add name=syslogremote target=remote remote=10.2.7.99 remote-port=514
/system logging add action=syslogremote topics=info
/system logging add action=syslogremote topics=warning
/system logging add action=syslogremote topics=critical
```

Métricas: Mikrotik no es compatible de forma directa con Node Exporter, usaremos la siguiente guía[1][2] más adelante, cuando tengamos que hacer los archivos docker-compose.yml y prometheus.yml. Esta configuración nos permitirá obtener métricas y además nos brinda un archivo ".json" con un Dashboard listo para Mikrotik.

[1] https://github.com/Jesof/mikrotik-exporter
[2] https://grafana.com/grafana/dashboards/24875-mikrotik-router-monitoring/

2, 3 y 4. Servidor FOG (10.2.7.5) (Linux, Apache y MariaDB)

- **Logs (Syslog):** Instala rsyslog si no viene por defecto y configúralo para reenviar todo a la central:

```
sudo apt install rsyslog -y
echo "*.* @10.2.7.99:514" | sudo tee -a /etc/rsyslog.d/50-default.conf
sudo systemctl restart rsyslog
```

Al final del archivo he añadido esto:

```
*.*@10.2.7.99:514
```



¿Qué hace? Esto es una regla de **centralización de logs**. Le dice a Rsyslog que copie todos los logs a la dirección 10.2.7.99:514.

Recolectores de Métricas

Ahora debemos configurar la recolección de métricas en las distintas VMs y servicios.

Preparación en la VM de FOG (10.2.7.5)

FOG no expone métricas nativas para Grafana, por lo que necesitamos instalar un "agente" que recoja los datos del sistema (CPU, RAM, Disco, Red) y de los servicios (Apache/MySQL).

A. Instalar Node Exporter

1. Instalarlo mediante “apt” (Debian/Ubuntu):

```
sudo apt update && sudo apt install prometheus-node-exporter -y
```

2. Debemos asegurarnos de que el puerto **9100** esté abierto en el firewall para que la VM de Grafana pueda leer los datos.
3. Comprobación de “**/etc/default/prometheus-node-exporter**”.

Como más adelante vamos a configurar alertas de servicios de **systemd**, debemos configurar este archivo. Debemos incluir **ARGS="--collector.systemd**” al final de este.

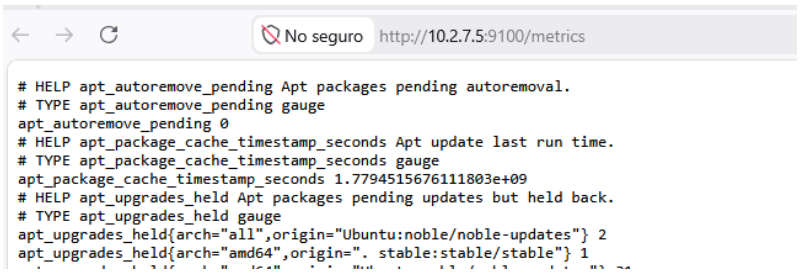
```
  GNU nano 7.2                                  /etc/default/prometheus-node-exporter *
# Set the command-line arguments to pass to the server.
# Due to shell escaping, to pass backslashes for regexes, you need to double
# them (\\d for \d). If running under systemd, you need to double them again
# (\\\\d to mean \d), and escape newlines too.
ARGS="--collector.systemd"
```

Ilustración 1 Modificación de "/etc/default/prometheus-node-exporter"

Y reiniciar: `sudo systemctl restart prometheus-node-exporter`

Podemos ver las métricas que recolecta desde esta URL:

```
http://10.2.7.5:9100/metrics
```



Configurar el Exportador de MariaDB (MySQL)

Paso A: Crear el usuario en MariaDB

Entra en la consola de tu base de datos:

```
sudo mysql -u root
```

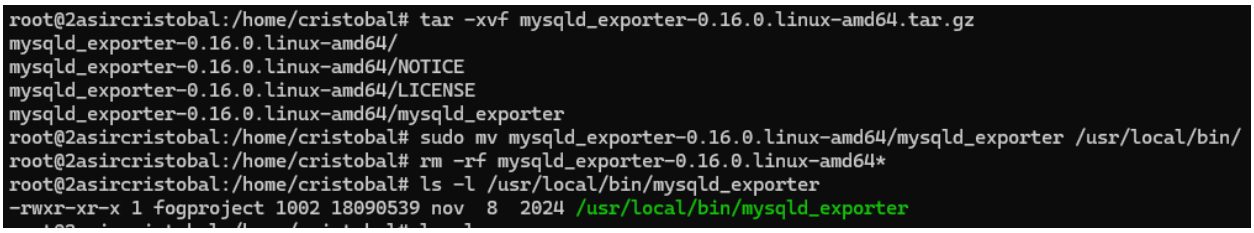
Dentro de MariaDB, copia y pega estos comandos (cambia la password por la contraseña que elijas):

```
CREATE USER 'exporter'@'localhost' IDENTIFIED BY 'password';
GRANT PROCESS, REPLICATION CLIENT, SELECT ON *.* TO 'exporter'@'localhost';
GRANT SLAVE MONITOR ON *.* TO 'exporter'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

Paso B: Descargar e instalar el binario

Descarga la herramienta oficial, descomprímela y muévela a la carpeta de ejecutables de Linux:

```
wget https://github.com/prometheus/mysqld_exporter/releases/download/v0.16.0/mysqld_exporter-0.16.0.linux-amd64.tar.gz
tar -xvf mysqld_exporter-0.16.0.linux-amd64.tar.gz
sudo mv mysqld_exporter-0.16.0.linux-amd64/mysqld_exporter /usr/local/bin/
rm -rf mysqld_exporter-0.16.0.linux-amd64*
```



Paso C: Crear el servicio en el sistema

Abre un archivo nuevo con el editor de texto:

```
sudo nano /etc/systemd/system/mysqld_exporter.service
```

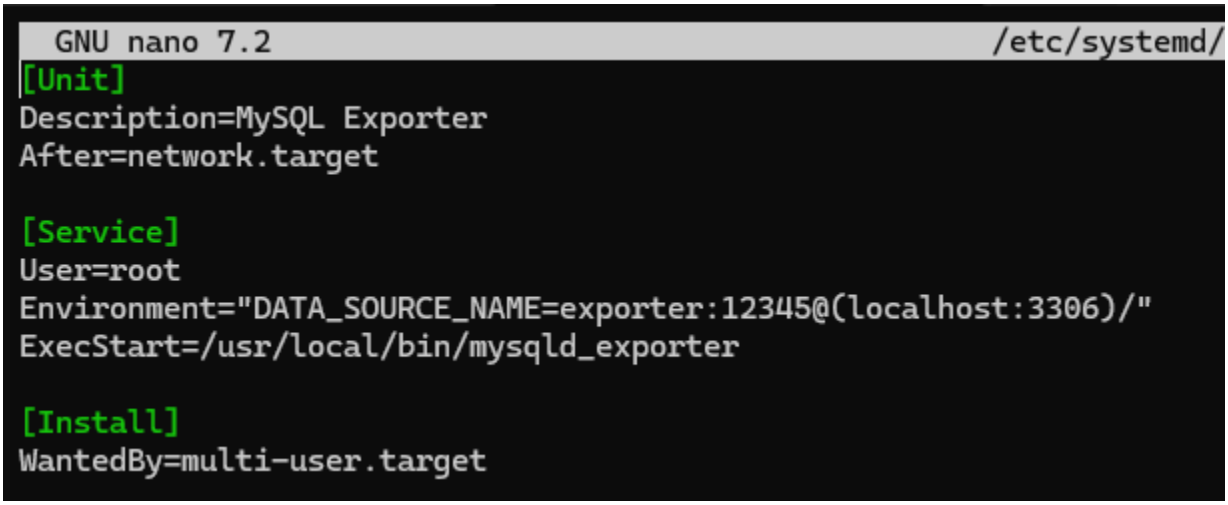
Pega este bloque de código dentro (si cambiaste la palabra password en el Paso A, cámbiala también aquí en DATA_SOURCE_NAME):

```
[Unit]
Description=MySQL Exporter
After=network.target

[Service]
User=root
Environment="DATA_SOURCE_NAME=exporter:password@(localhost:3306)/"
ExecStart=/usr/local/bin/mysqld_exporter

[Install]
WantedBy=multi-user.target
```

Guarda y cierra el archivo.

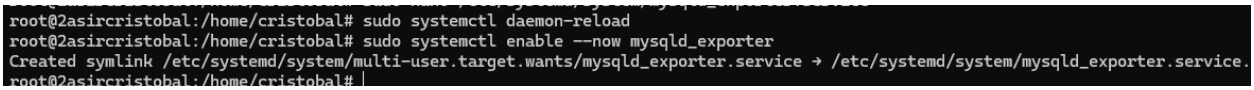


Paso D: Iniciar el exportador

Activa el servicio para que arranque automáticamente con la máquina:

```
sudo systemctl daemon-reload

sudo systemctl enable --now mysqld_exporter
```



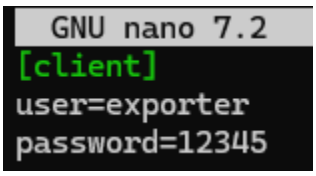
Paso E: Credenciales

Es posible que tengamos que crear un archivo con las credenciales con las que el node se conecte a la base de datos porque no consigue conectarse.

```
nano /etc/mysql/monitorización.cnf
```

Pondremos las credenciales del usuario que hemos creado en MariaDB.

```
[client]
user=exporter
password=12345
```



Ahora debemos modificar una línea para que el node busque este archivo cuando arranque.

```
nano /etc/systemd/system/mysqld_exporter.service
```

Ponemos esta línea:

```
ExecStart=/usr/local/bin/mysqld_exporter --config.my-cnf=/etc/mysql/monitorización.cnf
```

```
 GNU nano 7.2                                    /etc/systemd/system/mysqld_exporte
[Unit]
Description=MySQL Exporter
After=network.target

[Service]
User=root
Environment="DATA_SOURCE_NAME=exporter:12345@(localhost:3306)/"
#ExecStart=/usr/local/bin/mysqld_exporter
ExecStart=/usr/local/bin/mysqld_exporter --config.my-cnf=/etc/mysql/monitorización.cnf

[Install]
WantedBy=multi-user.target
```

Recarga el servicio del node

```
sudo systemctl daemon-reload

sudo systemctl restart mysqld_exporter
```

Nada más crearlo ya nos aparecerá el archivo log en el servidor de monitorización: Fíjate en la fecha.

```
[root@server2asir usuario]$ls -l /var/log/BBDD_LOG.log
-rw-r--r-- 1 syslog syslog 12104 may 22 14:29 /var/log/BBDD_LOG.log
```

```
[root@server2asir usuario]$tail /var/log/BBDD_LOG.log
2026-05-22T14:29:30+00:00 2asircristobal mysqld_exporter[350569]: time=2026-05-22T14:29:30.712Z level=INFO source=mysqld_exporter.go:252 msg="Scrap
er enabled" scraper=global_variables
2026-05-22T14:29:30+00:00 2asircristobal mysqld_exporter[350569]: time=2026-05-22T14:29:30.712Z level=INFO source=mysqld_exporter.go:252 msg="Scrap
er enabled" scraper=global_variables
2026-05-22T14:29:30+00:00 2asircristobal mysqld_exporter[350569]: time=2026-05-22T14:29:30.712Z level=INFO source=mysqld_exporter.go:252 msg="Scrap
er enabled" scraper=slave_status
```

Configurar el Exportador de Apache

Paso A: Activar mod_status en Apache

El exportador necesita que Apache muestre sus estadísticas internas de rendimiento. Actívalo con:

```
sudo a2enmod status

sudo systemctl restart apache2
```

```
root@2asircristobal:/home/cristobal# sudo a2enmod status
Module status already enabled
root@2asircristobal:/home/cristobal# sudo systemctl restart apache2
```

Paso B: Descargar e instalar el binario

Descarga el paquete del exportador de Apache, descomprímelo y muévelo a su sitio:

```
wget https://github.com/Lusitaniae/apache_exporter/releases/download/v1.0.8/apache_exporter-1.0.8.linux-amd64.tar.gz
tar -xvf apache_exporter-1.0.8.linux-amd64.tar.gz
sudo mv apache_exporter-1.0.8.linux-amd64/apache_exporter /usr/local/bin/
rm -rf apache_exporter-1.0.8.linux-amd64*
```

```
root@2asircristobal:/home/cristobal# tar -xvf apache_exporter-1.0.8.linux-amd64.tar.gz
apache_exporter-1.0.8.linux-amd64/
apache_exporter-1.0.8.linux-amd64/apache_exporter
apache_exporter-1.0.8.linux-amd64/LICENSE
root@2asircristobal:/home/cristobal# sudo mv apache_exporter-1.0.8.linux-amd64/apache_exporter /usr/local/bin/
root@2asircristobal:/home/cristobal# rm -rf apache_exporter-1.0.8.linux-amd64*
root@2asircristobal:/home/cristobal# ls -l /usr/local/bin/apache_exporter
-rwxr-xr-x 1 fogproject docker 15669525 jun 26  2024 /usr/local/bin/apache_exporter
root@2asircristobal:/home/cristobal#
```

Paso C: Crear el servicio en el sistema

Abre el editor para crear su servicio:

```
sudo nano /etc/systemd/system/apache_exporter.service
```

Pega estas líneas exactas dentro del archivo:

```
[Unit]
Description=Apache Exporter
After=network.target

[Service]
User=root
ExecStart=/usr/local/bin/apache_exporter --scrape_uri="http://localhost/server-status?auto"

[Install]
WantedBy=multi-user.target
```

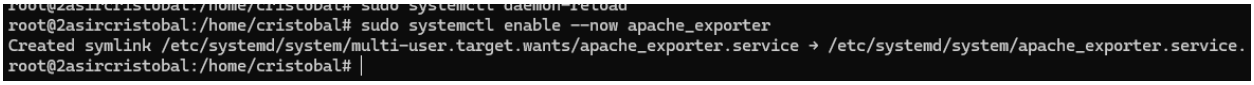


Guarda y cierra

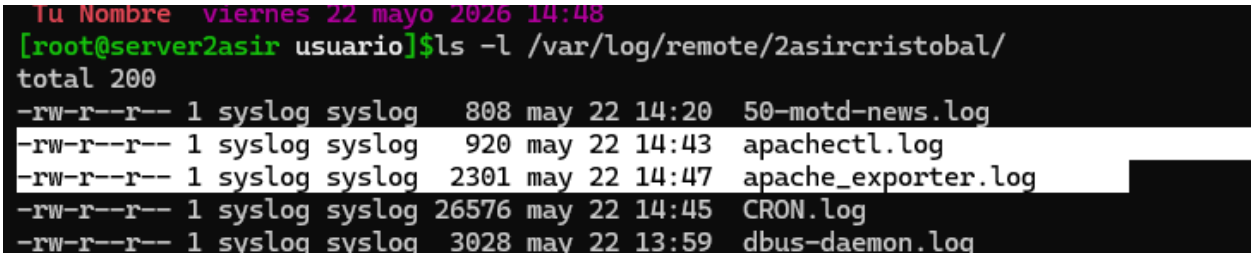
Paso D: Iniciar el exportador

Aplica los cambios e inicia el servicio de Apache Exporter:

```
sudo systemctl daemon-reload
sudo systemctl enable --now apache_exporter
```



Ya nos aparecen en el directorio de rsyslog en la VM de monitorización.



```
[root@server2asir usuario]$cat /var/log/remote/2asircristobal/apache_exporter.log
2026-05-22T14:47:55+00:00 2asircristobal apache_exporter[352951]: ts=2026-05-22T14:47:55.383Z caller=apache_exporter.go:65 level=info msg="Starting
 apache_exporter" version="(version=1.0.8, branch=HEAD, revision=887a69584e567f5f94de6b2f8bfde023ba7e697a)"
2026-05-22T14:47:55+00:00 2asircristobal apache_exporter[352951]: ts=2026-05-22T14:47:55.383Z caller=apache_exporter.go:65 level=info msg="Starting
 apache_exporter" version="(version=1.0.8, branch=HEAD, revision=887a69584e567f5f94de6b2f8bfde023ba7e697a)"
2026-05-22T14:47:55+00:00 2asircristobal apache_exporter[352951]: ts=2026-05-22T14:47:55.383Z caller=apache_exporter.go:66 level=info msg="Build co
ntext" build="(go=go1.22.4, platform=linux/amd64, user=root@e8819c63b929, date=20240626-14:49:16, tags=netgo)"
```

Comprobación final de puertos

Comprueba que los tres servicios están escuchando por sus puertos. Ejecuta este comando:

```
sudo ss -tlpn | grep -E "9100|9104|9117"
```

Si todo ha ido bien, verás tres líneas en la pantalla que confirman que los puertos **9100** (Node), **9104** (MariaDB) y **9117** (Apache) están activos en modo LISTEN.

```
root@2asircristobal:/home/cristobal# sudo ss -tlpn | grep -E "9100|9104|9117"
LISTEN 0      4096               *:9100             *:*    users:(("prometheus-node",pid=857,fd=3))

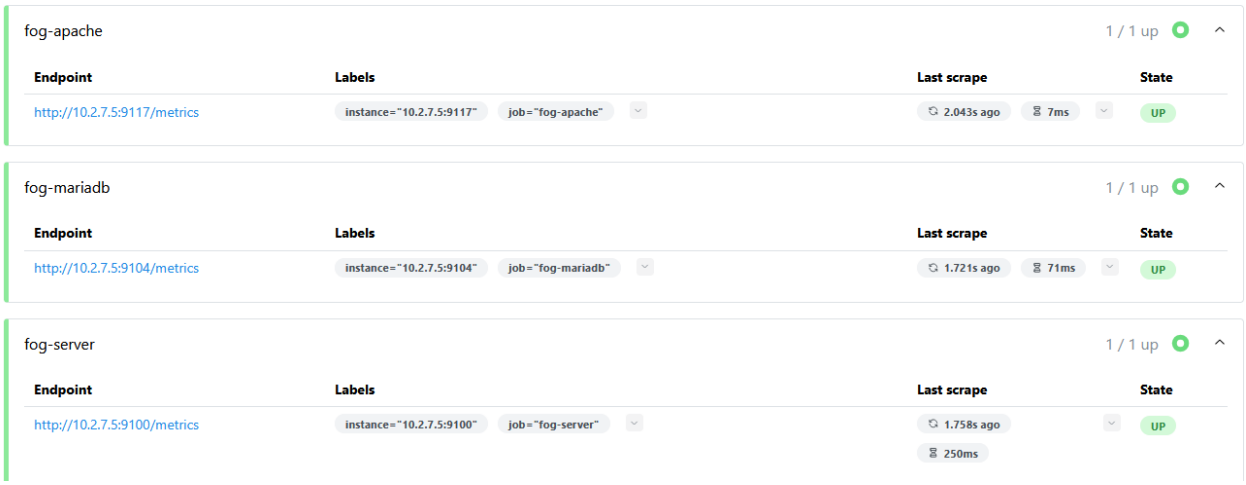
LISTEN 0      4096               *:9104             *:*    users:(("mysqld_exporter",pid=350569,fd=3))

LISTEN 0      4096               *:9117             *:*    users:(("apache_exporter",pid=352951,fd=3))

root@2asircristobal:/home/cristobal#
```

También podemos comprobar que Prometheus lo recibe los datos de manera adecuada desde:

```
http://10.2.7.99:9090/targets
```



5. Cliente Ubuntu

Paso 1: Logs del Sistema (Syslog)

Ubuntu ya trae rsyslog instalado por defecto. Solo tenemos que decirle que envíe una copia de todo al servidor central.

1.Editar la configuración de Rsyslog:En el cliente Ubuntu.

Abre el archivo de configuración con privilegios:

```
sudo nano /etc/rsyslog.d/50-default.conf
```

2.Añadir la regla de reenvío:Al final del archivo.

Ve al final de todo el documento y añade la siguiente línea (usando @ para UDP, tal como lo tienes en el servidor FOG):

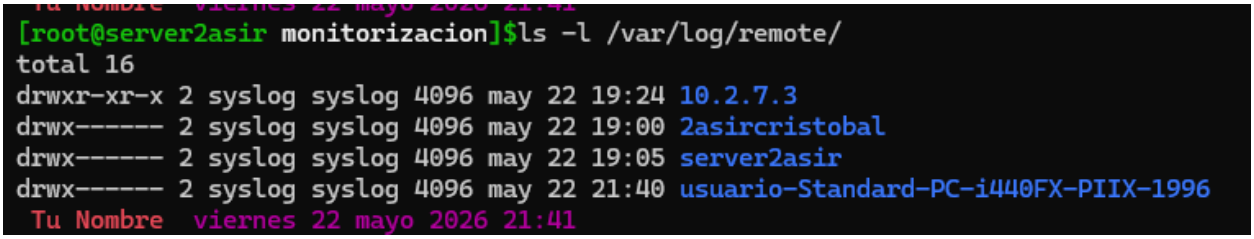
```
*.* @10.2.7.99:514
```

3.Reiniciar el servicio:Aplicar cambios.

Para que rsyslog empiece a transmitir los eventos en tiempo real, reinicia el demonio:

```
sudo systemctl restart Rsyslog
```

En el servidor de Monitorización ya lo muestra: es el último.



Paso 2: Métricas de Hardware (Node Exporter)

Instalamos el “Node Exporter” de igual manera que hicimos en el servidor FOG:

```
sudo apt update

sudo apt install prometheus-node-exporter -y
```

Nos aseguramos de que el servicio esté activo y configurado para iniciar con el sistema

```
sudo systemctl enable --now prometheus-node-exporter
```

Verificación del puerto: Por defecto, Node Exporter en Linux escucha en el puerto 9100. Puedes comprobar que está levantado ejecutando: **sudo ss -tlnp | grep 9100**

```
[root@usuario-Standard-PC-i440FX-PIIX-1996 usuario]$sudo ss -tlnp | grep 9100
LISTEN 0      4096               *:9100             *:*    users:(("prometheus-node",pid=3920,fd=3))
 Tu Nombre  viernes 22 mayo 2026 23:43
```

Para que transmita también los servicios del cliente, tendremos que hacer como en el servidor FOG:

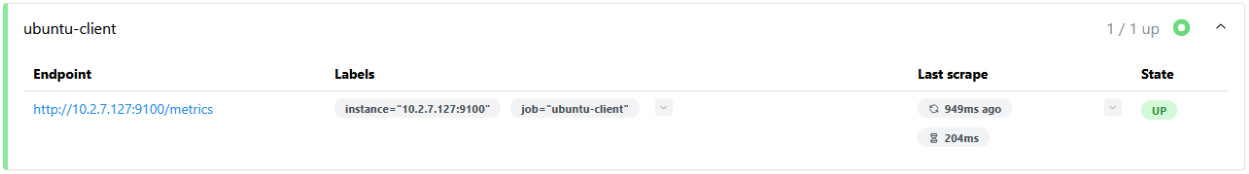
Comprobación de “**/etc/default/prometheus-node-exporter**”.

Como más adelante vamos a configurar alertas de servicios de systemd, debemos configurar este archivo. Debemos incluir **ARGS="--collector.systemd"** al final de este.

```
#ARGS=""
ARGS="--collector.systemd"
```

Y reiniciar: **sudo systemctl restart prometheus-node-exporter**

Comprobamos en Prometheus



Paso 3: Despliegue de la Monitorización en 10.2.7.99 (Docker Compose)

Crea un directorio en el servidor de monitorización, por ejemplo "**/monitorizacion**". En el debes crear los siguientes archivos:

Como estoy aprovechando lo hecho en el proyecto, a los servicios que teníamos antes (**prometheus, grafana y blackbox-exporter**), le he añadido los contenedores de **Loki** (el motor que almacenará los logs) y **Promtail** (el agente encargado de leer el archivo de logs del sistema y enviárselo a Loki).

Además, en el servicio **promtail**, mapearemos el directorio **/var/log** del servidor físico para que el contenedor pueda leer el archivo **ALERTA_TOTAL.log** que generará Rsyslog.

Configuración del router Mikrotik

Por último, para la monitorización del hardware **del router Mikrotik** hemos seguido los pasos necesarios que vienen en esta guía y hemos usado también el contenedor "**mikrotik_exporter**":

https://grafana.com/grafana/dashboards/24875-mikrotik-router-monitoring/

https://github.com/jesof/mikrotik-exporter

Lo cual luego nos permitirá crear en Grafana un Dashboard importando el correspondiente archivo "**.json**".

En el **router Mikrotik** hemos tenido que configurar un usuario/grupo y además habilitar el servicio API en el puerto por defecto (8728). Para ello desde la terminal del router hacemos:

```
# 1. Habilitar el servicio API en el puerto por defecto (8728)
/ip service set api address=0.0.0.0/0 disabled=no port=8728

# 2. Crear un grupo con permisos exclusivos de lectura para monitorización
/user group add name=monitoring policy=api,read

# 3. Crear el usuario 'prometheus' asociado al grupo y con la contraseña del docker
/user add name=prometheus group=monitoring password=12345
```

docker-compose.yml

Volvemos ahora al servidor de monitorización (10.2.7.99) y creamos el archivo:

```
services:
 prometheus:
  image: prom/prometheus:latest
  container_name: prometheus
  volumes:
   - ./prometheus.yml:/etc/prometheus/prometheus.yml
  ports:
   - "9090:9090"
  restart: unless-stopped

 grafana:
  image: grafana/grafana:latest
  container_name: grafana
  ports:
   - "3000:3000"
  environment:
   - GF_SECURITY_ADMIN_PASSWORD=12345
  volumes:
   - grafana-storage:/var/lib/grafana
  restart: unless-stopped

 blackbox-exporter:
  image: prom/blackbox-exporter:latest
  container_name: blackbox_exporter
  ports:
   - "9115:9115"
  restart: unless-stopped

 #
============================================================================
 # NUEVOS SERVICIOS PARA LA GESTIÓN DE LOGS (ACTIVIDAD 3)
 #
============================================================================

 loki:
  image: grafana/loki:latest
  container_name: loki
  ports:
   - "3100:3100"
  command: -config.file=/etc/loki/local-config.yaml
```

```
  restart: unless-stopped

 promtail:
  image: grafana/promtail:latest
  container_name: promtail
  volumes:
   - /var/log:/var/log # Enlace clave para acceder al ALERTA_TOTAL.log del host
   - ./promtail-config.yml:/etc/promtail/config.yml
  command: -config.file=/etc/promtail/config.yml
  restart: unless-stopped

# === NUEVO: Exportador nativo para MikroTik (API) ===
 mikrotik-exporter:
  image: ghcr.io/jesof/mikrotik-exporter:latest
  container_name: mikrotik_exporter
  ports:
   - "9094:9090" # Mapeamos su puerto interno al 9094 de la máquina física
  environment:
   - SERVER_ADDR=0.0.0.0:9090
   - ROUTERS_CONFIG=[{"name":"mikrotik-
router","address":"10.2.7.3:8728","username":"prometheus","password":"12345"}]
  restart: unless-stopped

volumes:
 grafana-storage:
```

prometheus.yml

Aprovechamos el archivo del proyecto y mantenemos el intervalo global de rastreo (5s) y la tarea original (fog-server), añadiremos las nuevas tareas para monitorizar los exportadores específicos (BBDD MariaDB, Servidor Web Apache, Cliente Windows) y el router Mikrotik.

```
global:
  scrape_interval: 5s

scrape_configs:
  #(Servidor FOG Linux)
  - job_name: 'fog-server'
    static_configs:
      - targets: ['10.2.7.5:9100']

  #
==========================================================================
  # NUEVAS TAREAS DE RECOLECCIÓN DE MÉTRICAS (ACTIVIDAD 3)
  #
==========================================================================

  # 1. Base de datos MariaDB (en el servidor FOG) usando exportador específico
  - job_name: 'fog-mariadb'
    static_configs:
      - targets: ['10.2.7.5:9104']

  # 2. Servidor Web Apache (en el servidor FOG) usando exportador específico
  - job_name: 'fog-apache'
    static_configs:
      - targets: ['10.2.7.5:9117']

  # 3. Cliente Ubuntu
  - job_name: 'ubuntu-client'
    static_configs:
      - targets: ['10.2.7.127:9100'] # IP de tu nuevo cliente Ubuntu

  # 4. Router Mikrotik
  - job_name: 'mikrotik-metrics'
    static_configs:
      - targets: ['mikrotik-exporter:9090'] # Apunta directamente al contenedor por su red
interna
```

promtail-config.yml

Archivo plano de texto llamado promtail-config.yml dentro del mismo directorio de trabajo (/root/monitorizacion/ o donde ejecutes el comando) con la configuración de lectura:

```
server:
  http_listen_port: 9080
  grpc_listen_port: 0

positions:
  filename: /tmp/positions.yaml

clients:
  - url: http://loki:3100/loki/api/v1/push

scrape_configs:
- job_name: rsyslog-alertas
  static_configs:
  - targets:
      - localhost
    labels:
      job: rsyslog_criticos
      __path__: /var/log/ALERTA_TOTAL.log # Archivo que rellenará Rsyslog localmente

## CONFIGURACIÓN PARA CUANDO MIKROTIK NO ENVIABA LOS LOGS A ALERTA_TOTAL.log
# === NUEVO JOB PARA EXPONER MIKROTIK ===
- job_name: mikrotik-error-critical
  static_configs:
  - targets:
      - localhost
    labels:
      job: mikrotik_errores
      dispositivo: mikrotik
      __path__: /var/log/remote/10.2.7.3/system,error,critical.log

- job_name: mikrotik-warnings
  static_configs:
  - targets:
      - localhost
    labels:
      job: mikrotik_warnings
      dispositivo: mikrotik
      __path__: /var/log/remote/10.2.7.3/script,warning.log
```

Regla de firewall

En nuestro caso teníamos configurado el Firewall por el Proyecto. **En el servidor FOG**.

1. Verificar el puerto: Entra por SSH al servidor FOG y comprueba si hay algún servicio escuchando en el puerto 9104:

```
sudo ss -tlnp | grep 9104

sudo ss -tlnp | grep 9117
```



2. Abrir el Firewall de FOG: Si el servicio está corriendo pero Prometheus no llega, abre el puerto en el cortafuegos de Linux ejecutando:

```
sudo ufw allow 9104/tcp

sudo ufw allow 9117/tcp

sudo ufw reload
```

Pasos finales recomendados en la máquina para ponerlo en marcha:

1. **Permisos del Log:** Asegúrate de que el archivo final de registros críticos existe en el sistema anfitrión y tiene permisos de lectura para que el Docker de Promtail lo pueda absorber sin restricciones. En el servidor de monitorización.

```
sudo touch /var/log/ALERTA_TOTAL.log

sudo chmod 644 /var/log/ALERTA_TOTAL.log
```

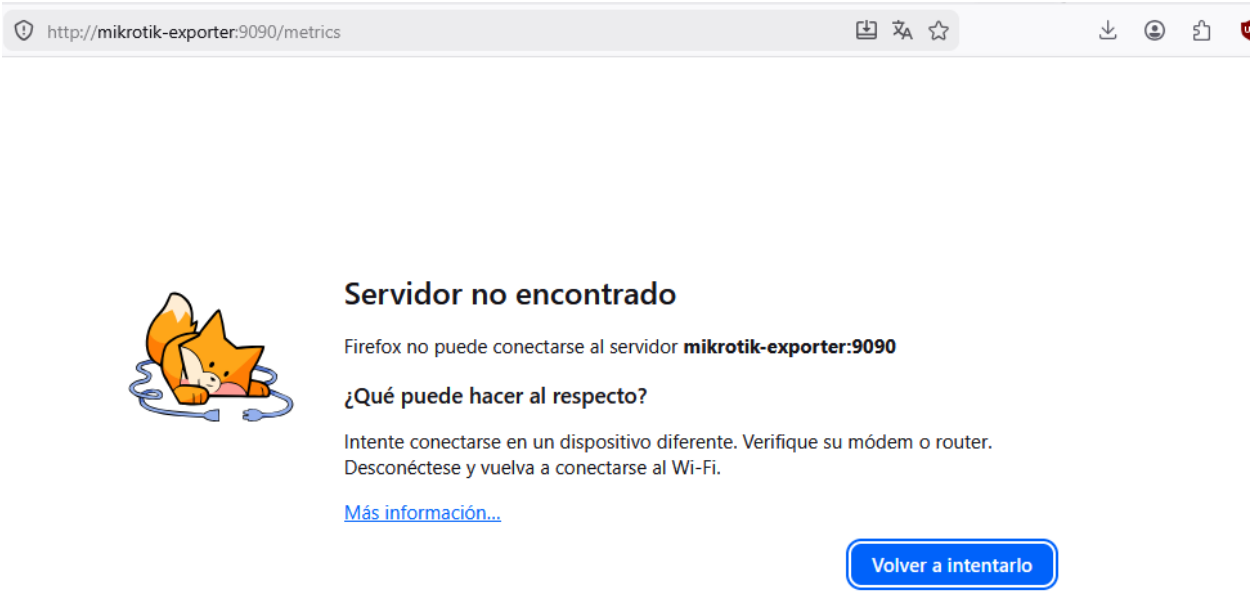
2. **Reaplicar el despliegue:** Detén e inicia de nuevo Docker Compose para reconstruir los contenedores con las nuevas dependencias de Loki y las configuraciones actualizadas:

```
docker compose down

docker compose up -d
```

Consideraciones

Cuando vayamos a http://10.2.7.99:9090/targets veremos que todos los targets están "UP", pero cuando vayamos a mirar los del node Mikrotik http://mikrotik-exporter:9090/metrics nos saldrá esta imagen en el navegador.



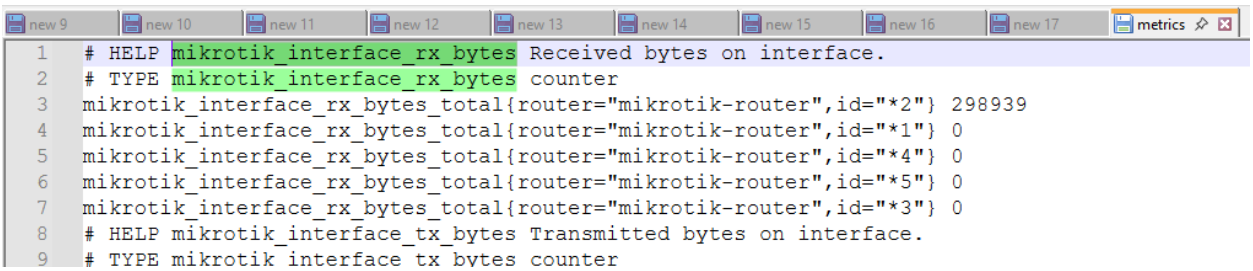
Eso se debe a que **Prometheus** puede resolver y llegar a ese nombre porque está dentro del mismo ecosistema de contenedores. Por eso en su panel aparece en verde (UP).

El navegador web está corriendo de forma externa. Tu sistema operativo no conoce qué significa la palabra **mikrotik-exporter** ni tiene esa ruta en sus servidores DNS locales, por lo que da un fallo de resolución de nombre.

Usa: http://10.2.7.99:9094/metrics

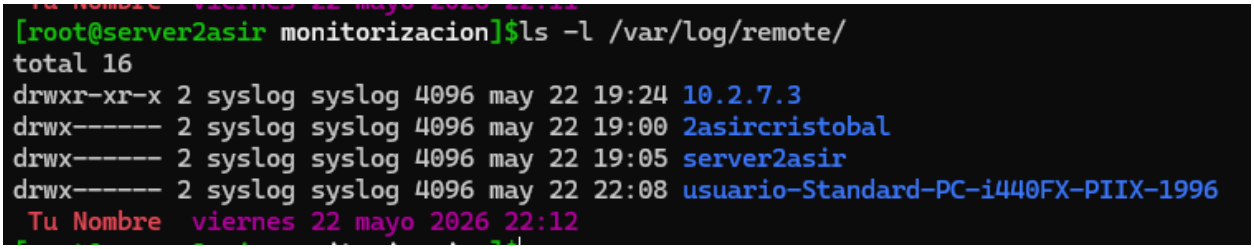
El puerto 9094 es el que le pusimos al router Mikrotik en el Docker Compose como puerto externo.

Generará un archivo "metrics" sin extensión.



Comprobación de los LOGS

Hemos establecido los directorios de los diferentes equipos en "**/var/log/remote/**".



Comprobamos el log "**ALERTA_TOTAL.log**"

```
tail -f /var/log/ALERTA_TOTAL.log
```

Para ello usamos comandos de prueba:

A. **En Servidor FOG:**

Desde el propio Servidor de Logs o el **Servidor FOG (Linux):**

```
logger -p user.crit "TEST: Alerta critica de prueba para el dashboard de Grafana"
```

Fallo de Apache:

```
logger -p local6.err -t apache2 "level=ERROR msg=\"PHP Fatal error: Allowed memory size of 536870912 bytes exhausted in /var/www/html/index.php\""
```

Fallo de MariaDB: l

```
logger -p local6.err -t mysqld "level=ERROR msg=\"InnoDB: Database page corruption on disk or a bug in the strategy\""
```

Simular un error crítico en el despliegue de FOG:

```
logger -p local6.err -t fog-project "level=ERROR msg=\"FOGImageReplicator: Configured storage node is not reachable or disk is full\""
```

B. **Desde el cliente**:

```
logger -p user.crit "LOG_UBUNTU: Alerta critica real desde el nuevo cliente"
```

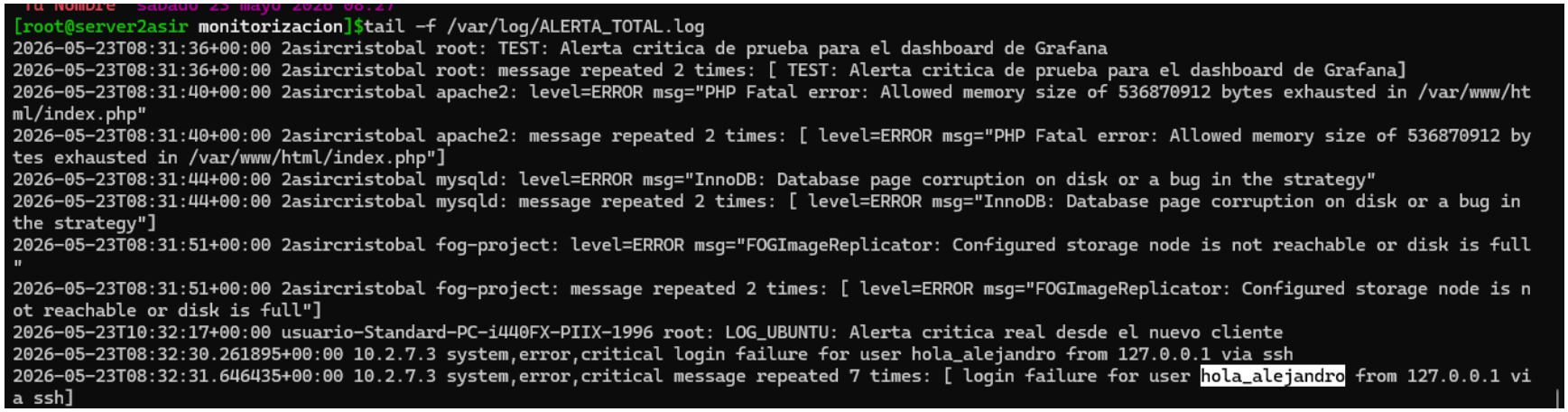
C. Router Mikrotik:

Simulamos un intento fallido de inicio de sesión.

```
[admin@MikroTik] > /system ssh address=127.0.0.1 user=hola_alejandro
password:
password:
password:
password:
password:
password:
password:
password:
password:

Welcome back!
07:12:27 echo: system,error,critical login failure for user hola_alejandro from 127.0.0.1 via ssh
07:12:27 echo: system,error,critical login failure for user hola_alejandro from 127.0.0.1 via ssh
07:12:28 echo: system,error,critical login failure for user hola_alejandro from 127.0.0.1 via ssh
07:12:29 echo: system,error,critical login failure for user hola_alejandro from 127.0.0.1 via ssh
07:12:30 echo: system,error,critical login failure for user hola_alejandro from 127.0.0.1 via ssh
07:12:30 echo: system,error,critical login failure for user hola_alejandro from 127.0.0.1 via ssh
07:12:31 echo: system,error,critical login failure for user hola_alejandro from 127.0.0.1 via ssh
07:12:32 echo: system,error,critical login failure for user hola_alejandro from 127.0.0.1 via ssh
[admin@MikroTik] > :log warning "TEST: Alerta critica desde MikroTik para Grafana"
[admin@MikroTik] >
```

Aparecerán en ALERTA_TOTAL.log

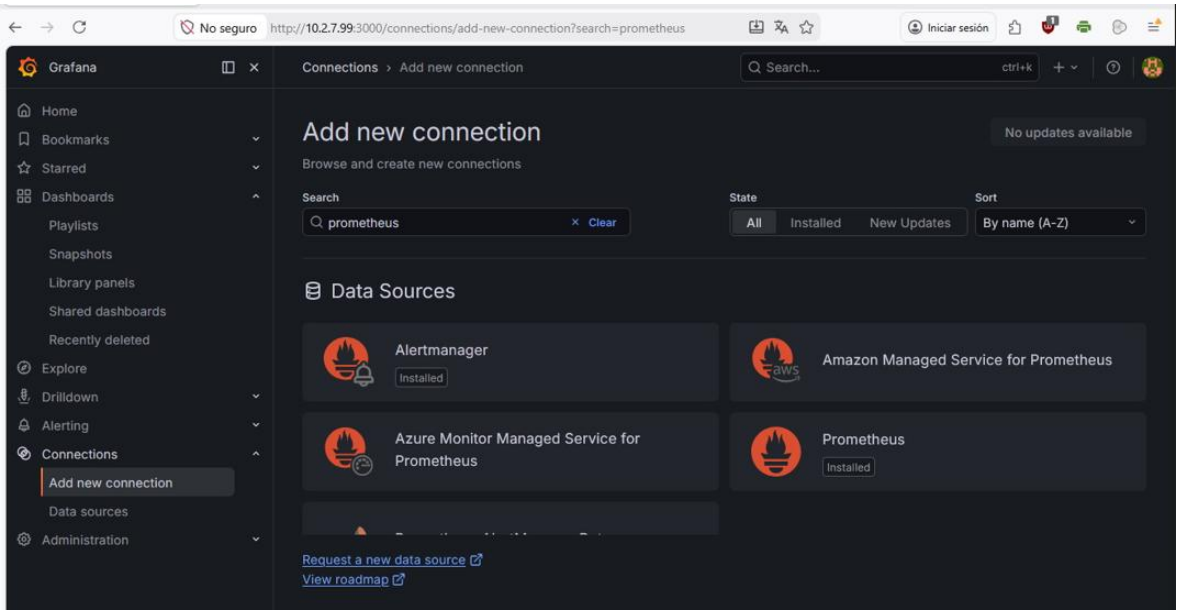


Paso 4: Grafana. Creación de Connections, Dashboards y Alertas

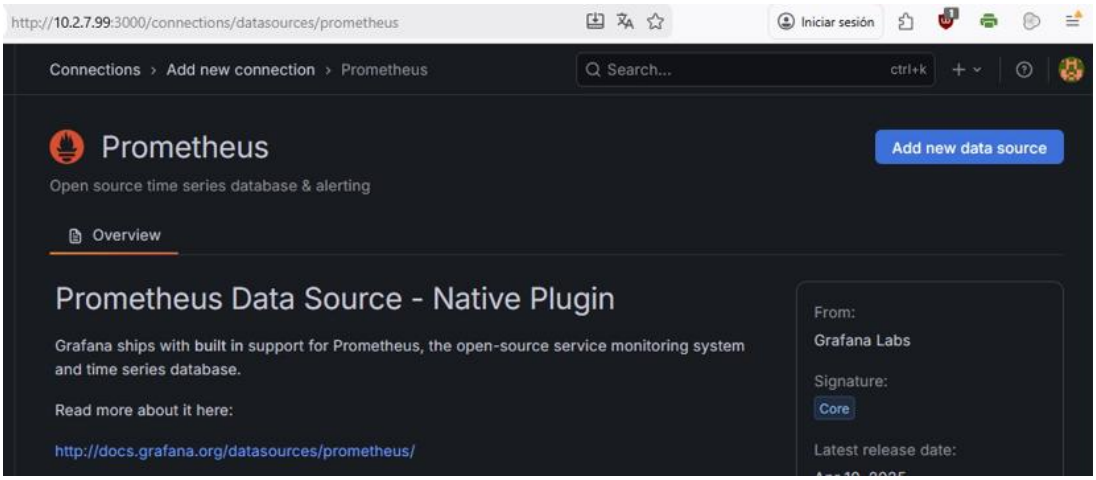
Entramos en Grafana: http://IP-DE-TU-VM-MONITOR:3000 → http://10.2.7.99:3000/

Añadir "Data Source"

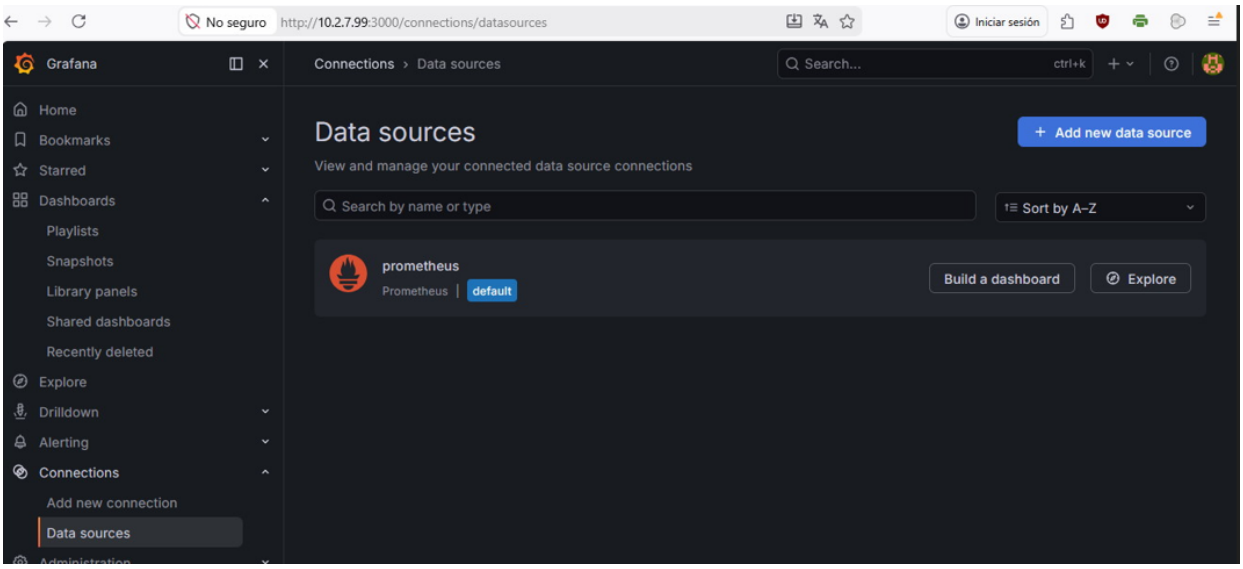
Ve a **Connections** > **Add new connection** →**Data Sources**. Busca "**Prometheus**" y añádelo.



Dale a "**Add new data source**".

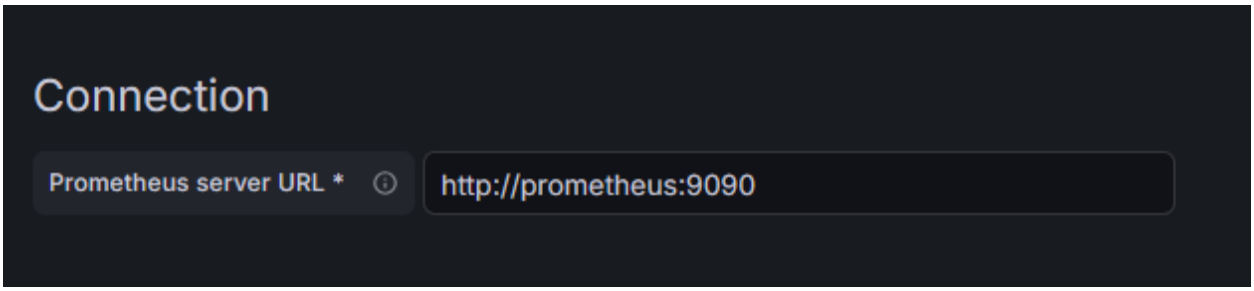


Una vez que lo has añadido, tendrás que configurarlo:

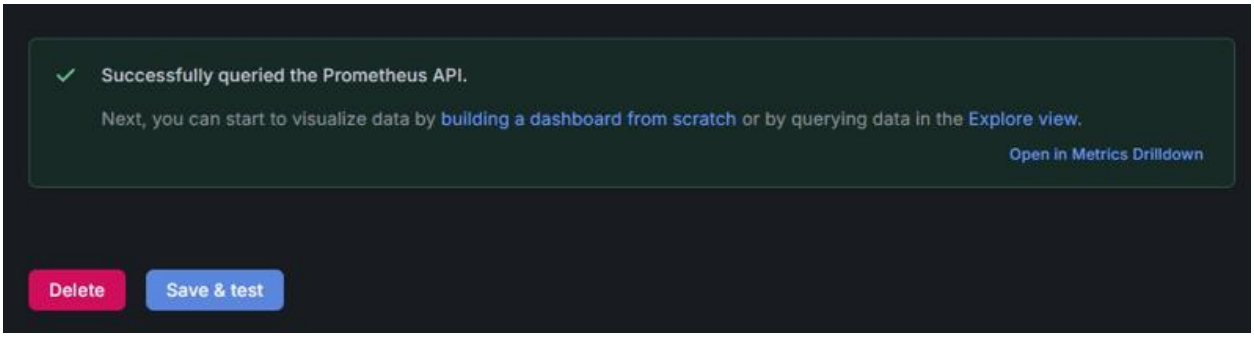


En la URL pon: http://prometheus:9090

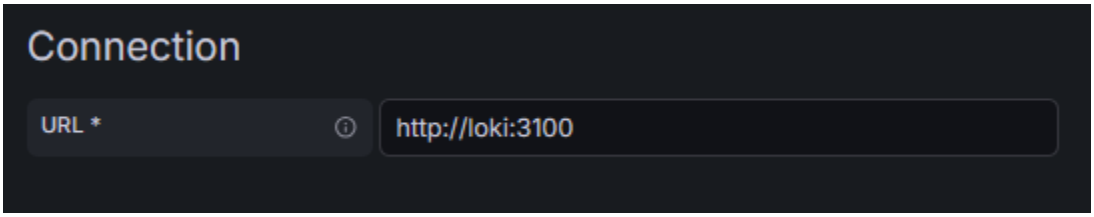
Es así porque ambos contenedores están en la misma red de Docker y pueden comunicarse por sus nombres (DNS).

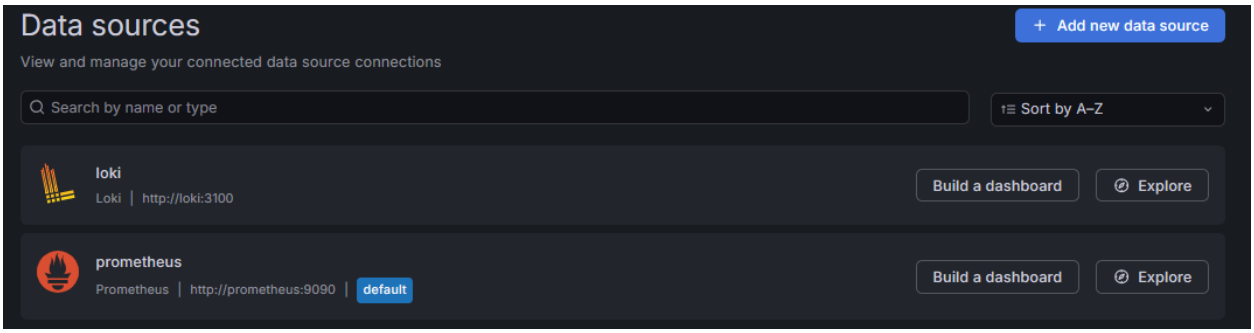


Para esta ocasión no es necesario configurar nada más. Haz clic en **Save & Test**.



RECUERDA: Debemos seguir los mismos pasos, pero buscando a Loki y poniendo su nombre de contenedor y su puerto.





Dasboards

Dashboards preconfigurados: Vamos a mostrar brevemente como se pueden usar Dashboards preconfigurados (archivos ".json") para el uso de Dashboads.

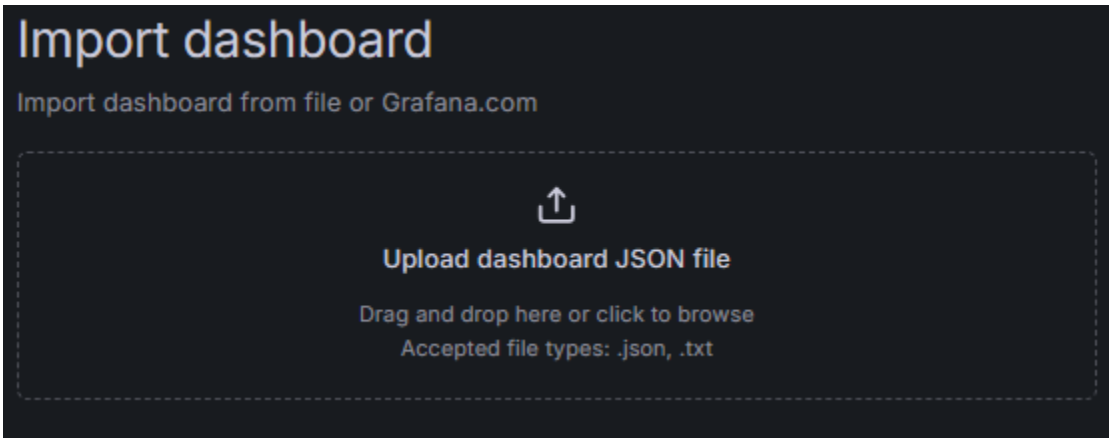
Para Equipos Linux:

Usamos el siguiente, nos servirá tanto para el Servidor FOG como para el cliente Ubuntu.

https://grafana.com/grafana/dashboards/1860-node-exporter-full/

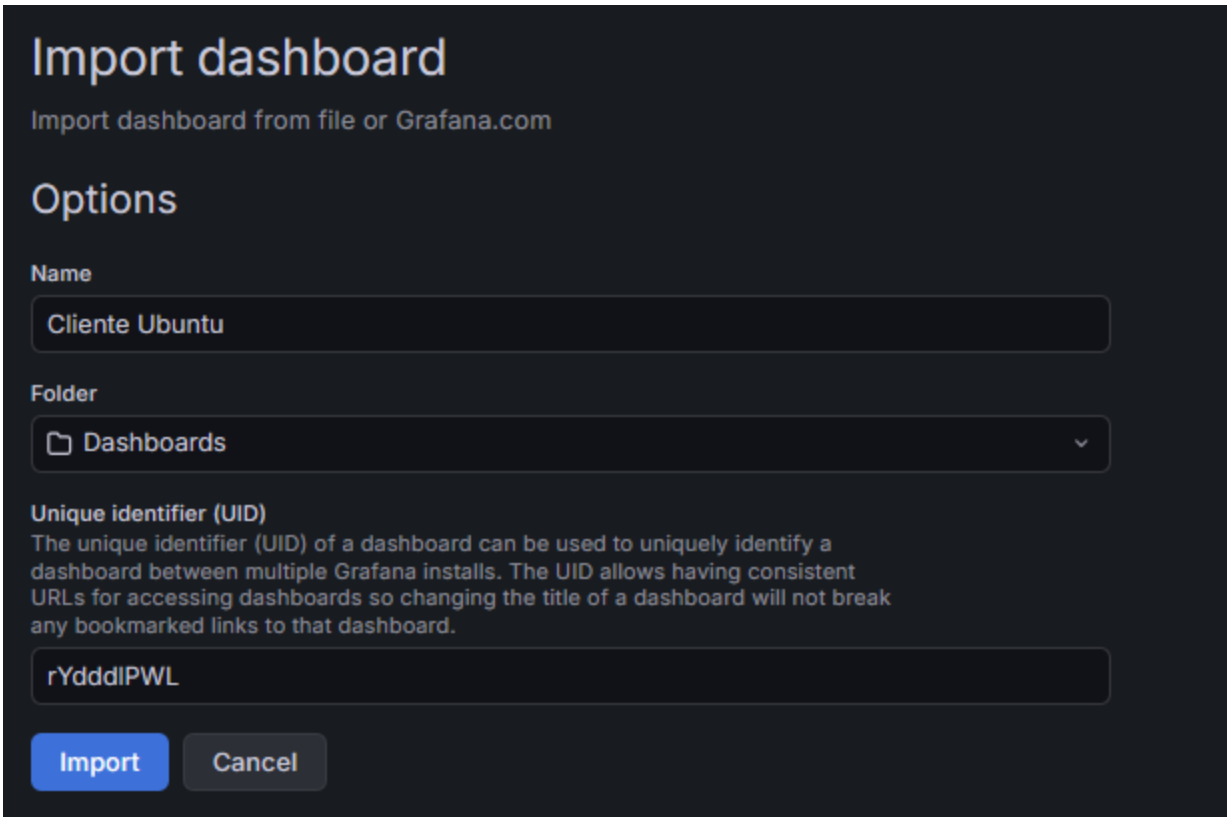
Vamos a Dashboard →New → Import

Y ahora a "Upload dashboard JSON file".

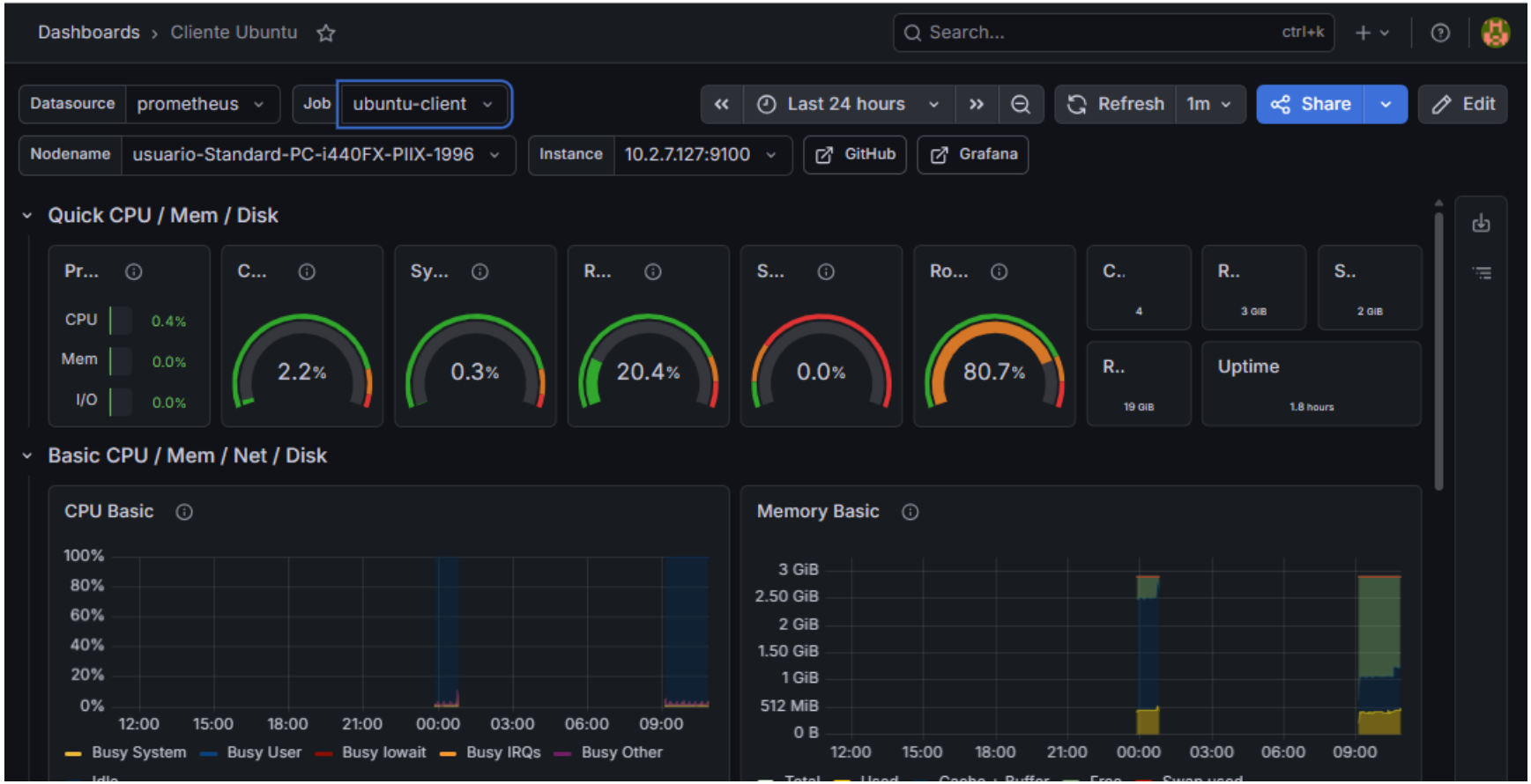


Simplemente importamos el archivo que nos hemos bajado en el enlace anterior.

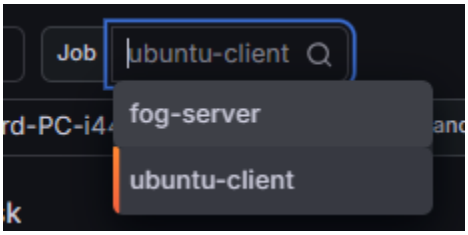
Establecemos un nombre y carpeta donde se guardará.


Import dashboard
Import dashboard from file or Grafana.com
Options
Name
Cliente Ubuntu
Folder
Dashboards
Unique identifier (UID)
The unique identifier (UID) of a dashboard can be used to uniquely identify a dashboard between multiple Grafana installs. The UID allows having consistent URLs for accessing dashboards so changing the title of a dashboard will not break any bookmarked links to that dashboard.
rYdddlPWL
Import
Cancel

Dashboard generado:



Ten en cuenta que al funcionar tanto para el Servidor FOG como para el Cliente Ubuntu, se puede cambiar para quien sirve en ese momento desde “Job”.



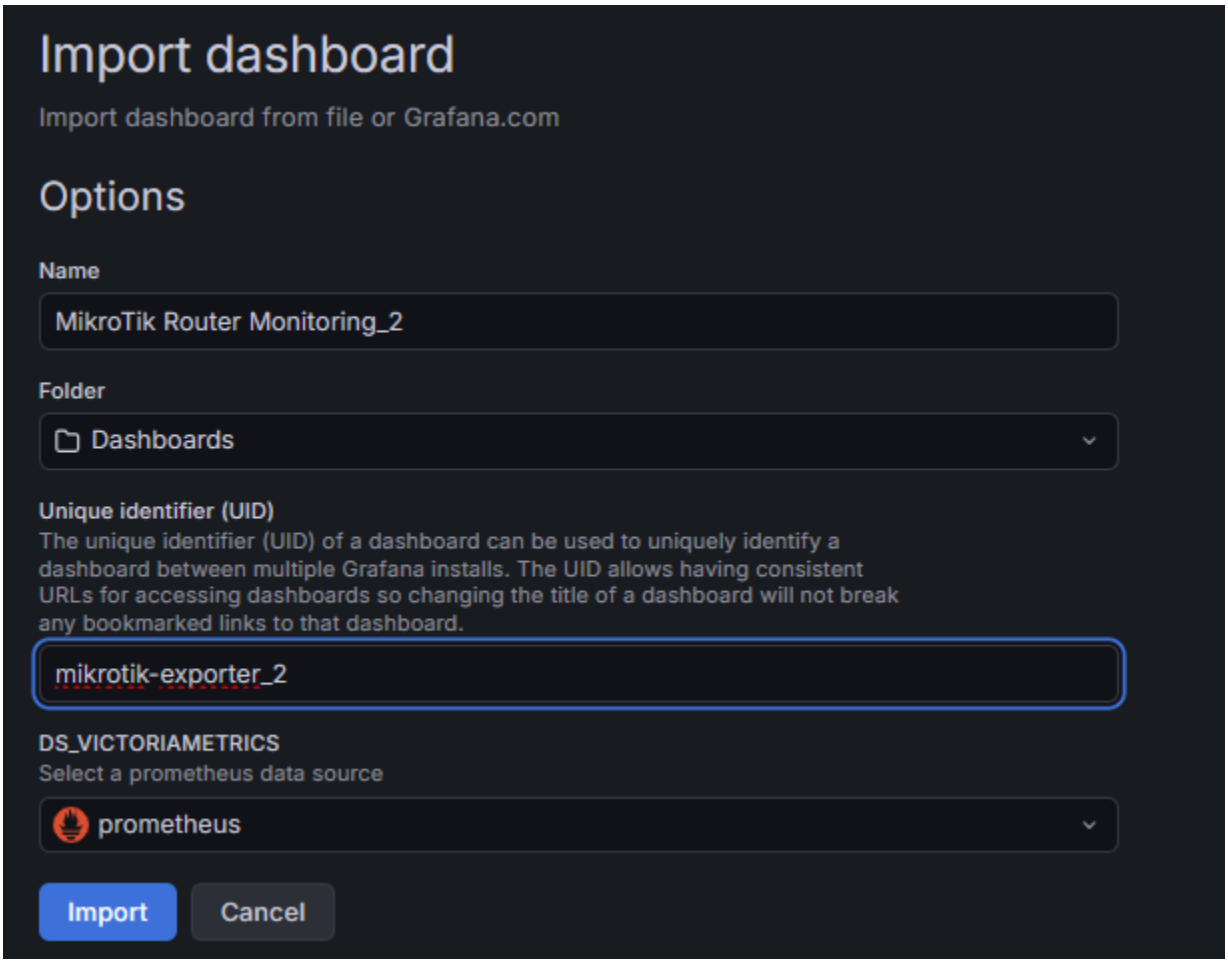
Para router Mikrotik:

Seguimos la siguiente guía:

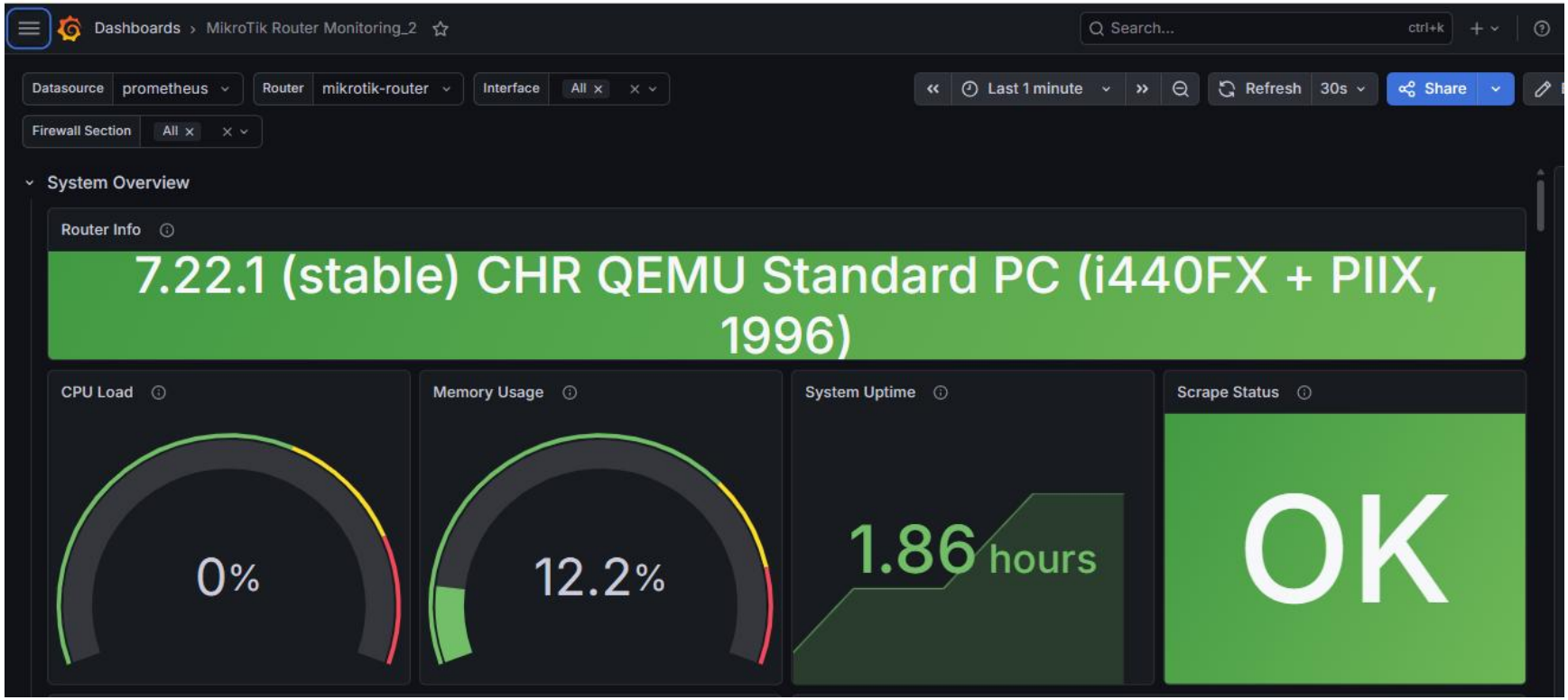
https://grafana.com/grafana/dashboards/24875-mikrotik-router-monitoring/

https://github.com/jesof/mikrotik-exporter

Como ya hemos configurado lo necesario en los puntos anteriores, ahora solo tendremos que importar el archivo. En esta ocasión si se indica que debemos establecer la Data Source:



Dashboard generado:

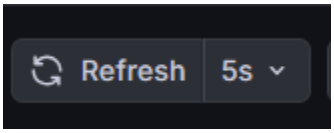


Dashboard personalizado

Una vez que tenemos configuradas las “Data Sources”.

Dashboard → New → New Dashboard

Configura el refresh para el tiempo que quieras. En mi caso 5 segundos.



Ahora vamos a meter paneles al Dashboard.

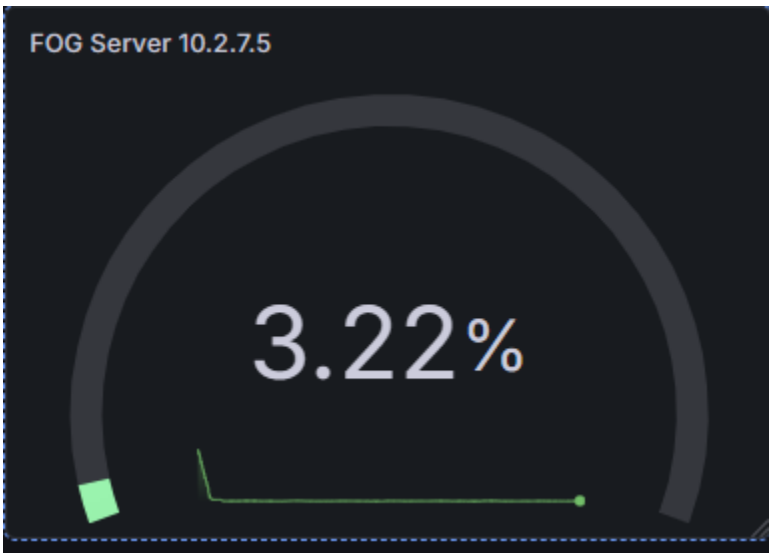
Servidor FOG:

Para las métricas vamos a usar siempre como **“Data Source”** a **Prometheus:**

CPU:

Query: **100 - (avg by (instance)**
(rate(node_cpu_seconds_total{instance="10.2.7.5:9100", mode="idle"}[5m])) * 100)

Tipo de Visualización: **Gauge**.

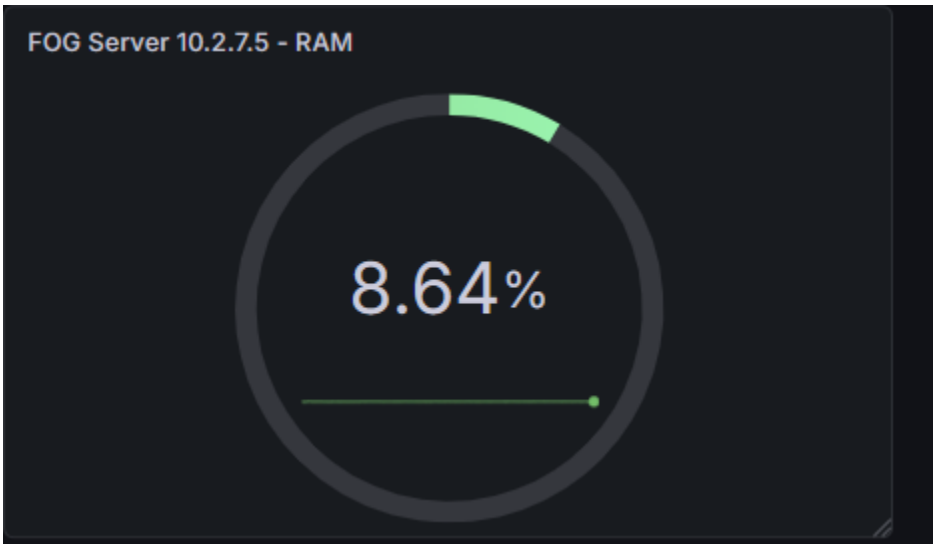


Memoria usada:

100 - ((node_memory_MemAvailable_bytes{instance="10.2.7.5:9100"} / node_memory_MemTotal_bytes{instance="10.2.7.5:9100"}) * 100)

Vista: Gauge.

Units → Misc → Percent



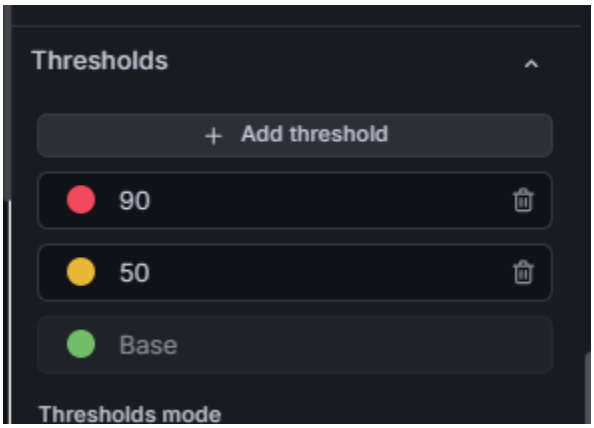
Almacenamiento usado:

(1 - (node_filesystem_free_bytes{instance="10.2.7.5:9100", mountpoint="/"} / node_filesystem_size_bytes{instance="10.2.7.5:9100", mountpoint="/"})) * 100

Unit → Data → Gigabytes

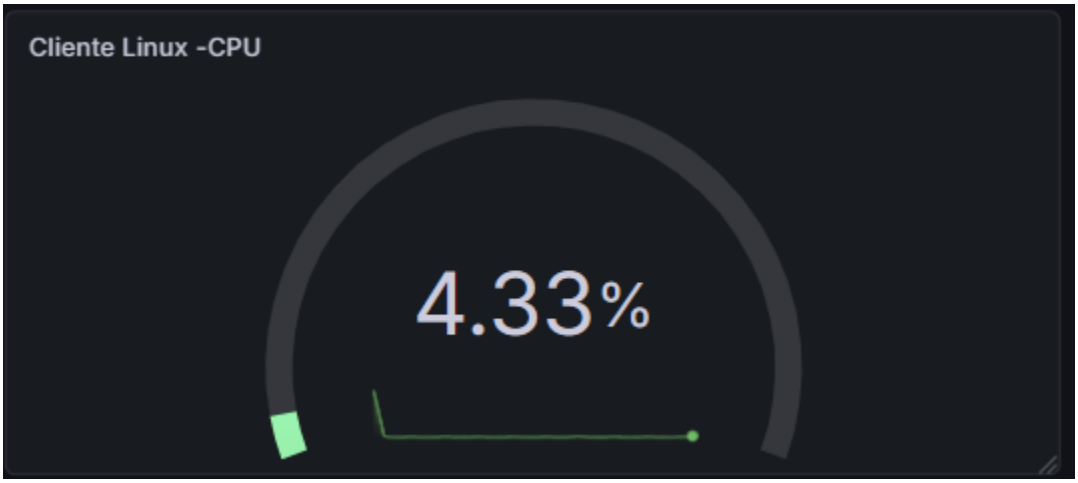


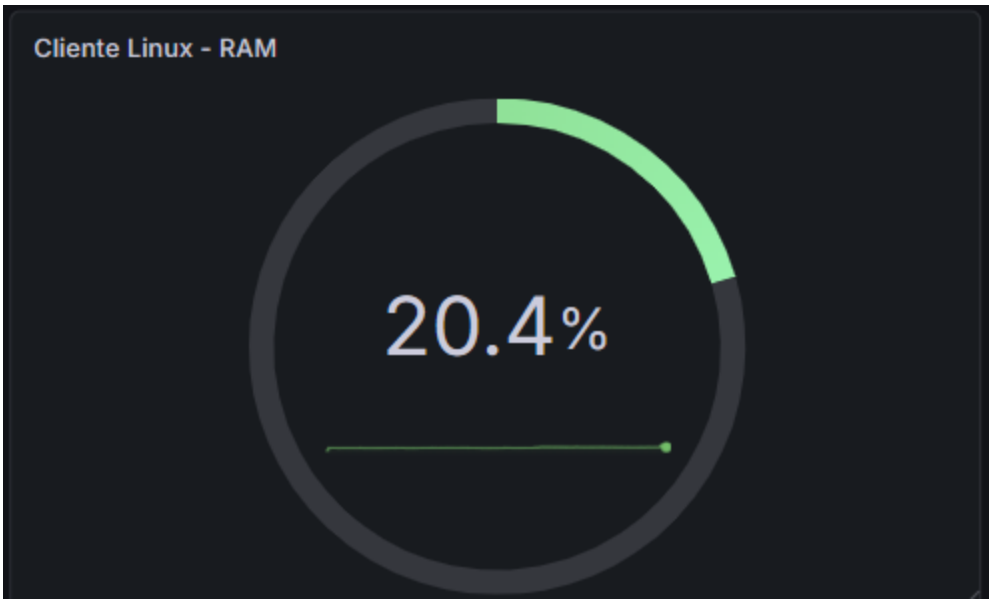
En el campo Thresolds podemos establecer colores de advertencia según el nivel de la métrica. Por ejemplo, para el uso de CPU, estableces naranja para 50% y rojo para 90%. Por debajo de 50% es verde.



Cliente Linux:

Prácticamente lo mismo que hemos visto antes, pero adaptándolo a su IP.




Cliente Linux - RAM
20.4%

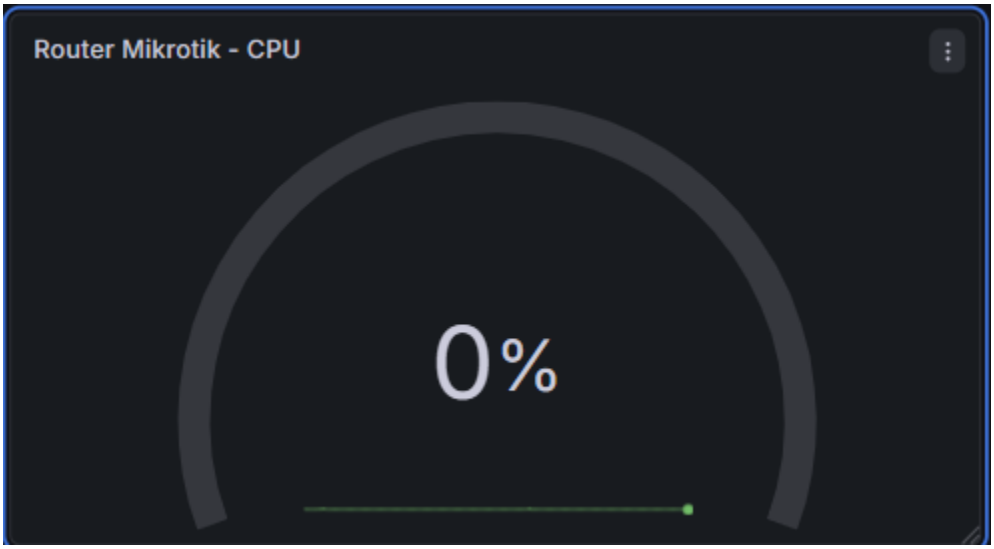

Cliente Linux - Almacenamiento
75.5%

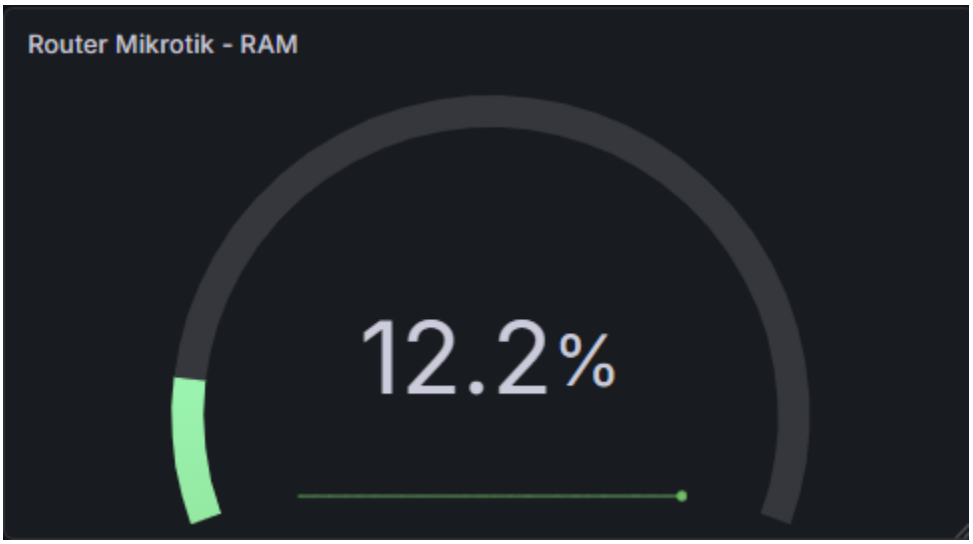
Router Mikrotik:

El router Mikrotik usa otro tipo de Query, pero por lo demás es igual.

CPU: mikrotik_system_cpu_load{router="mikrotik-router"}

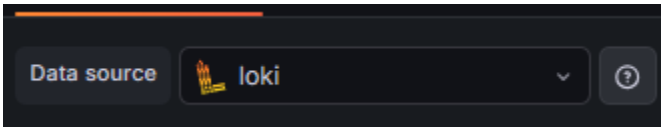
RAM: (1 - (mikrotik_system_free_memory_bytes{router="mikrotik-router"} / mikrotik_system_total_memory_bytes{router="mikrotik-router"})) * 100





LOGS

Parecido a los anteriores, pero en este caso usamos como **Data Source** a **Loki**.



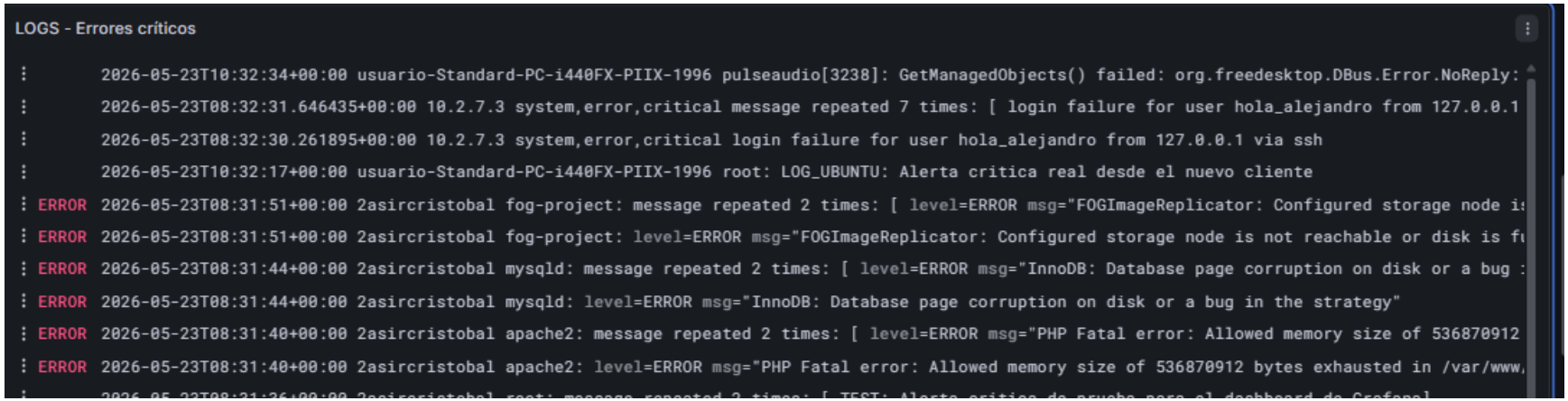
En el Query usaremos: **{job="rsyslog_criticos"}**

Que es el **job** que configuramos en el archivo **promtail-config.yml**

```
scrape_configs:
- job_name: rsyslog-alertas
  static_configs:
  - targets:
      - localhost
    labels:
      job: rsyslog_criticos
      __path__: /var/log/ALERTA_TOTAL.log # Archivo que rellenará Rsyslog localmente
```

En la vista usamos → **Logs**

Panel de LOGS.



Servicios

Apache

Ver si está arriba:

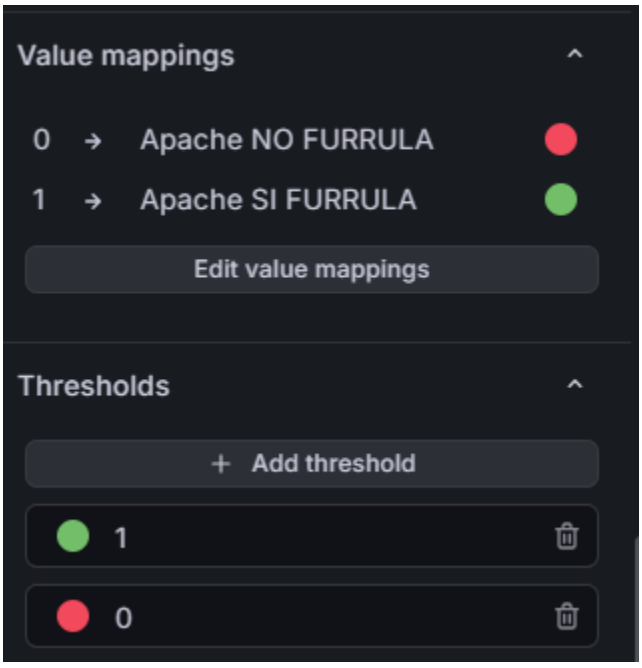
Data Source: Prometheus

Query: apache_up

Vista → Stat

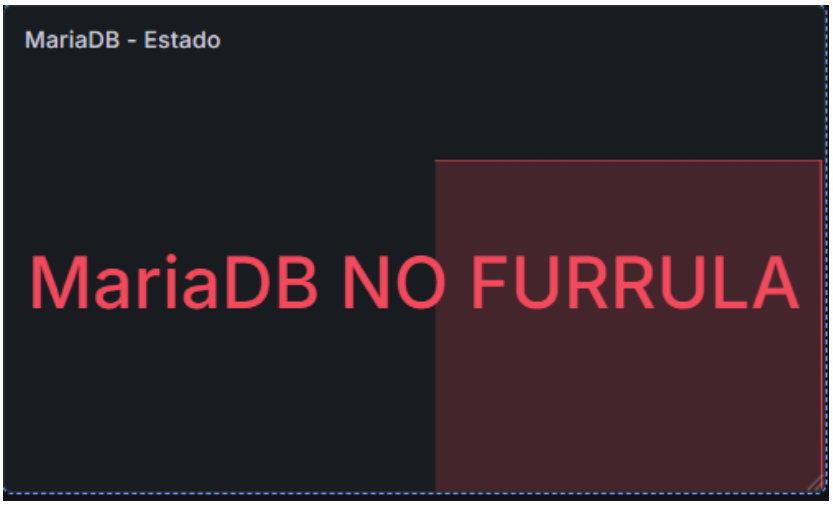


Puedes establecer colores y valores para mostrar texto.



MariaDB

Para MariaDB es algo parecido: Mostrando en este caso con el servicio apagado.



Servicios de FOG

Algo parecido hemos hecho con los servicios del FOG Project

FOGImageReplicator: (Réplica de imágenes entre servidores):

```
node_systemd_unit_state{name="FOGImageReplicator.service", state="active"}
```

FOGScheduler (Tareas programadas y colas de despliegue):

```
node_systemd_unit_state{name="FOGScheduler.service", state="active"}
```

De igual manera se puede hacer para

“**FOG Multicast Manager**”:

```
node_systemd_unit_state{name="FOGMulticastManager.service", state="active"}
```

“**FOG Ping Hosts**”:

```
node_systemd_unit_state{name="FOGPingHosts.service", state="active"}
```

“**FOG Snapin Hash**”.

```
node_systemd_unit_state{name="FOGSnapinHash.service", state="active"}
```

Tabla del Dashboard donde se agrupan los estados de los servicios.



Alertas de Eventos Críticos

Antes de crear la alerta, debemos decirle a Grafana a dónde enviar el mensaje. En nuestro caso usaremos Telegram.

Paso 1: Crear el Bot de Telegram

Antes de ir a Grafana, necesitamos crear el "túnel" de mensajería:

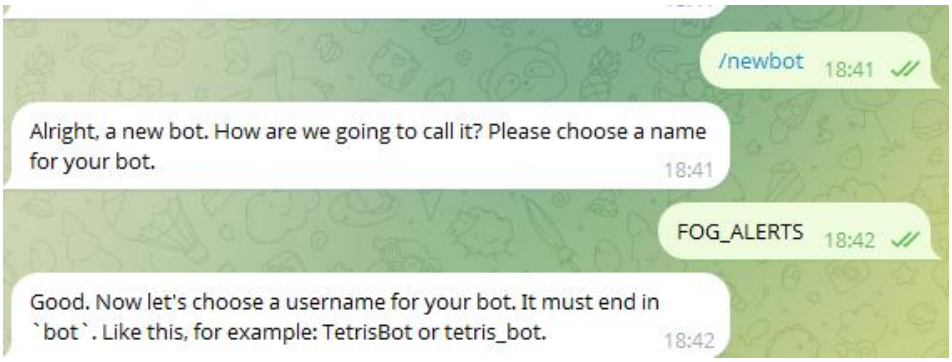
1. Busca a **@BotFather** en Telegram y envíale el comando **/newbot**.



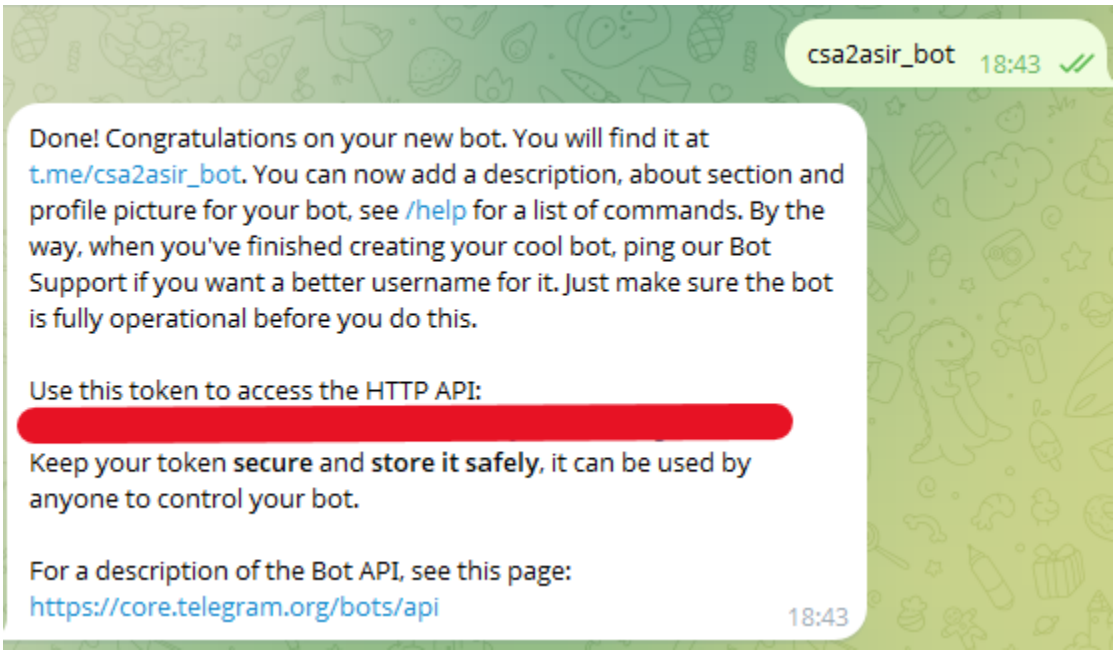
Ilustración 2 Telegram @BotFather 01



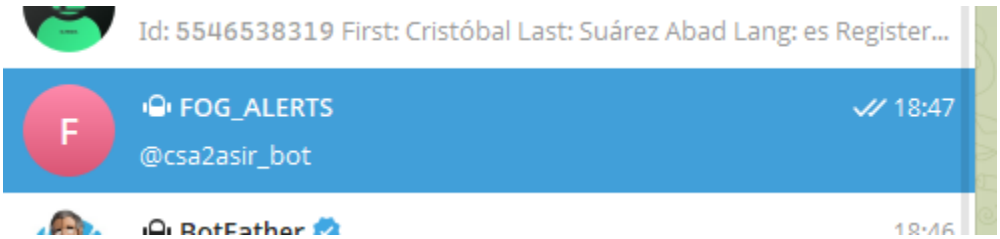
2. Sigue las instrucciones (ponle nombre y usuario).

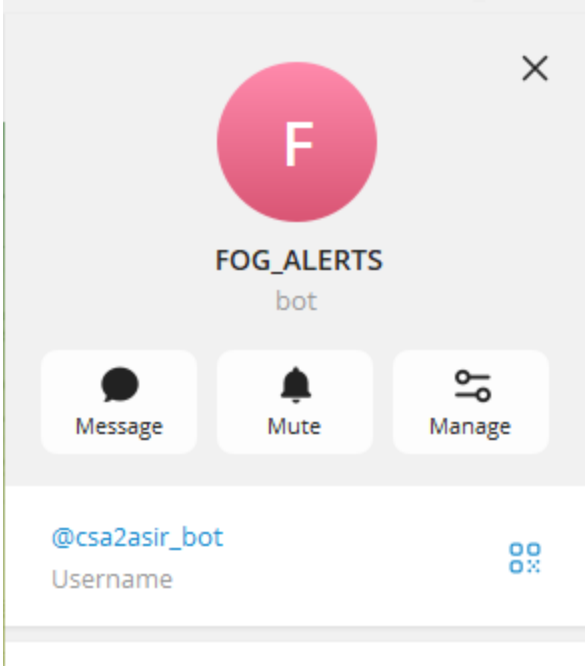


3. **Guarda el API TOKEN** que te darán (se ve algo como 7012345678:AAH...).

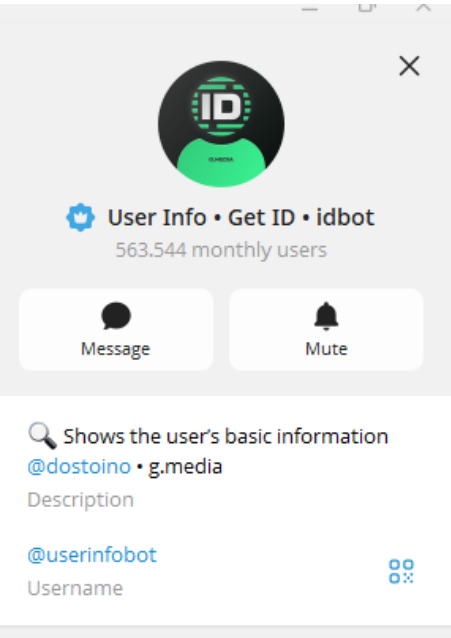


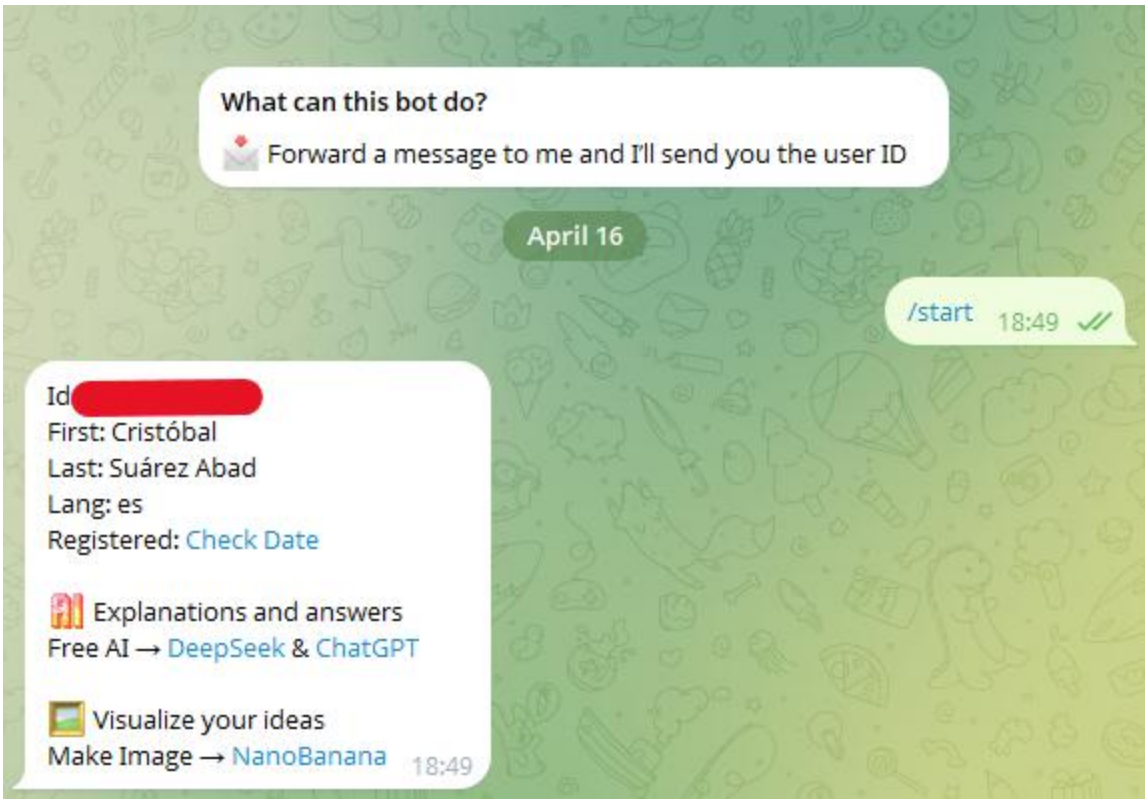
4. Busca tu nuevo bot en Telegram y dale a **/start** (esto es vital).





5. **Obtén tu Chat ID**: Reenvía un mensaje tuyo al bot @userinfobot y él te dirá tu ID numérico (ej: 123456789).

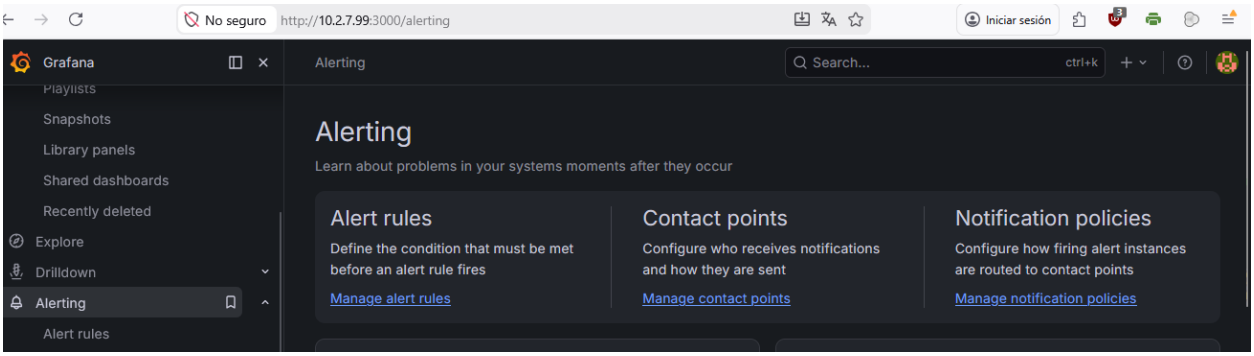



What can this bot do?
Forward a message to me and I'll send you the user ID
April 16
/start 18:49
Id [redacted]
First: Cristóbal
Last: Suárez Abad
Lang: es
Registered: Check Date
Explanations and answers
Free AI → DeepSeek & ChatGPT
Visualize your ideas
Make Image → NanoBanana 18:49

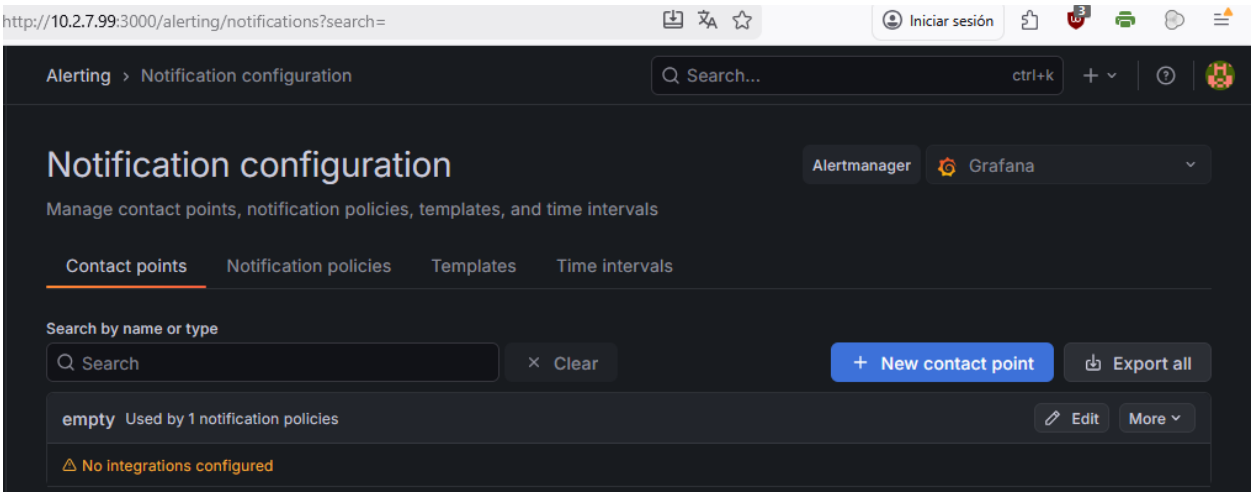
Paso 2: Configurar el Punto de Contacto en Grafana

Ahora volvemos a Grafana:

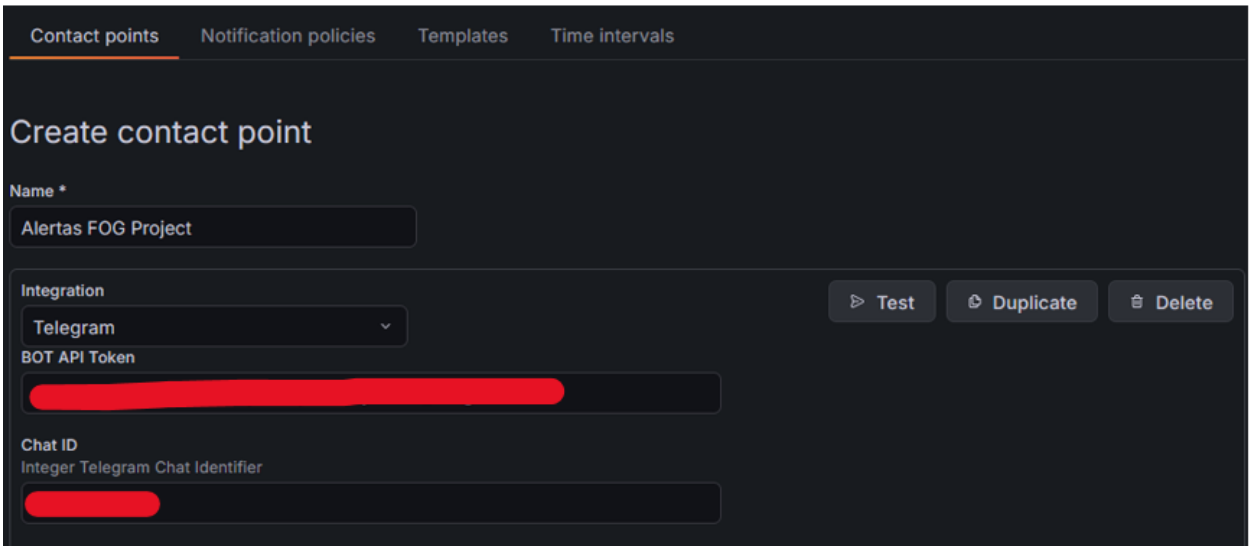
1. En el menú lateral de Grafana, ve a **Alerting → Contact points**.



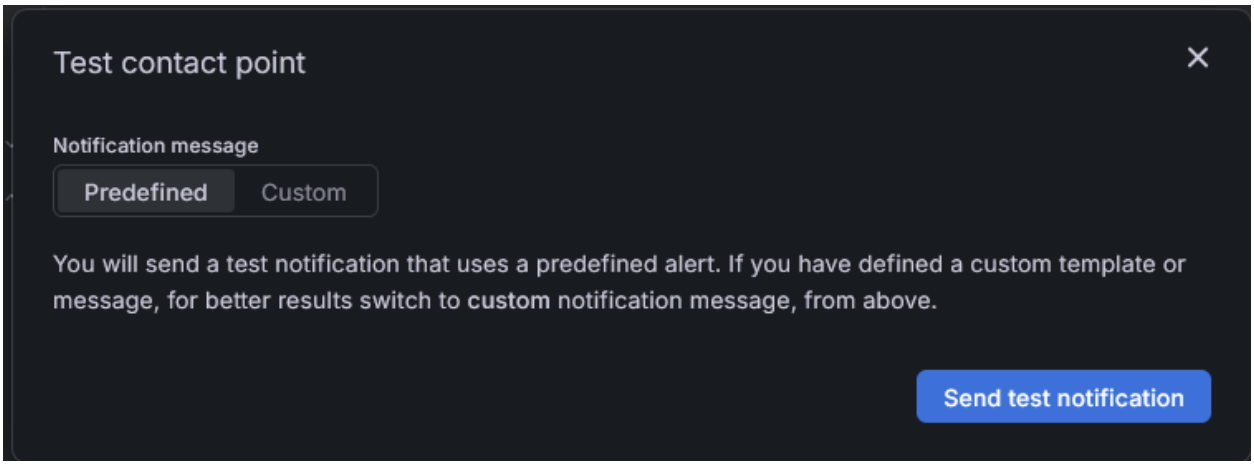
2. Haz clic en **New Contact Point**.

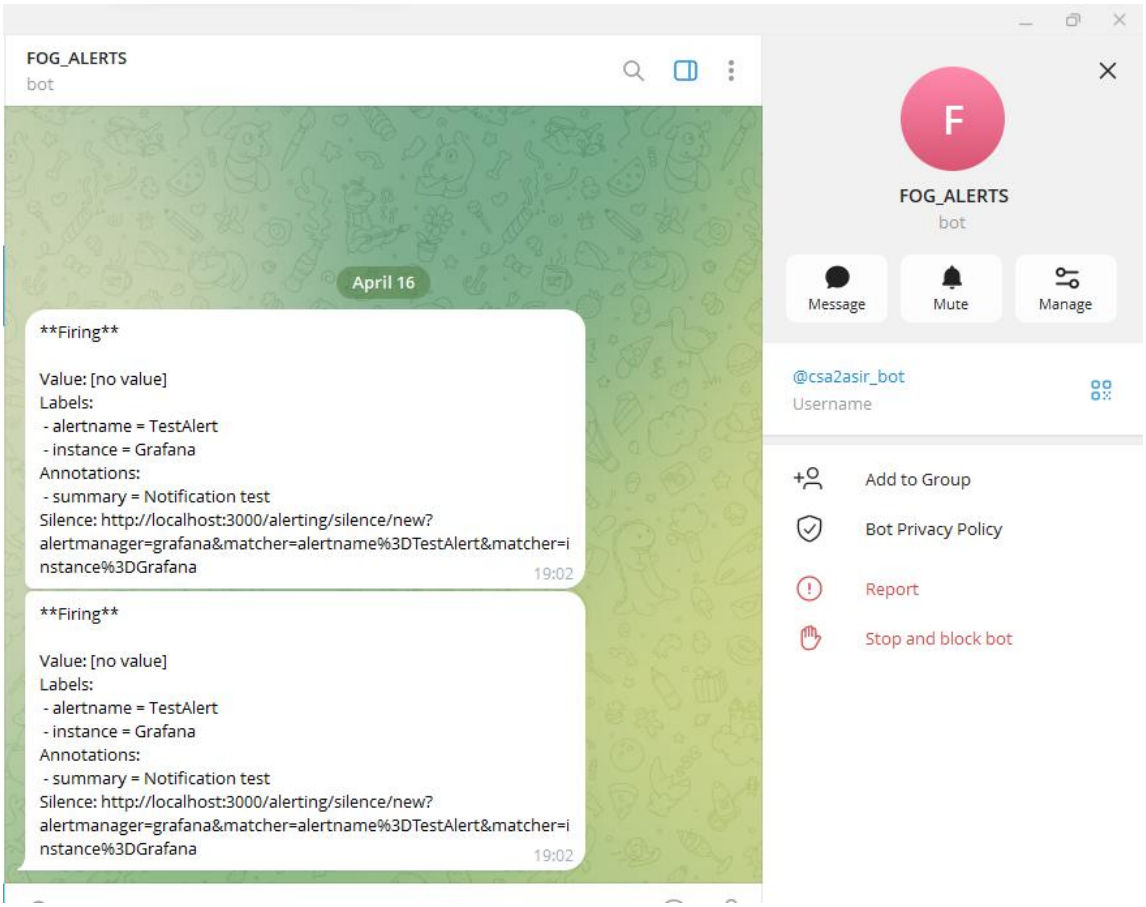


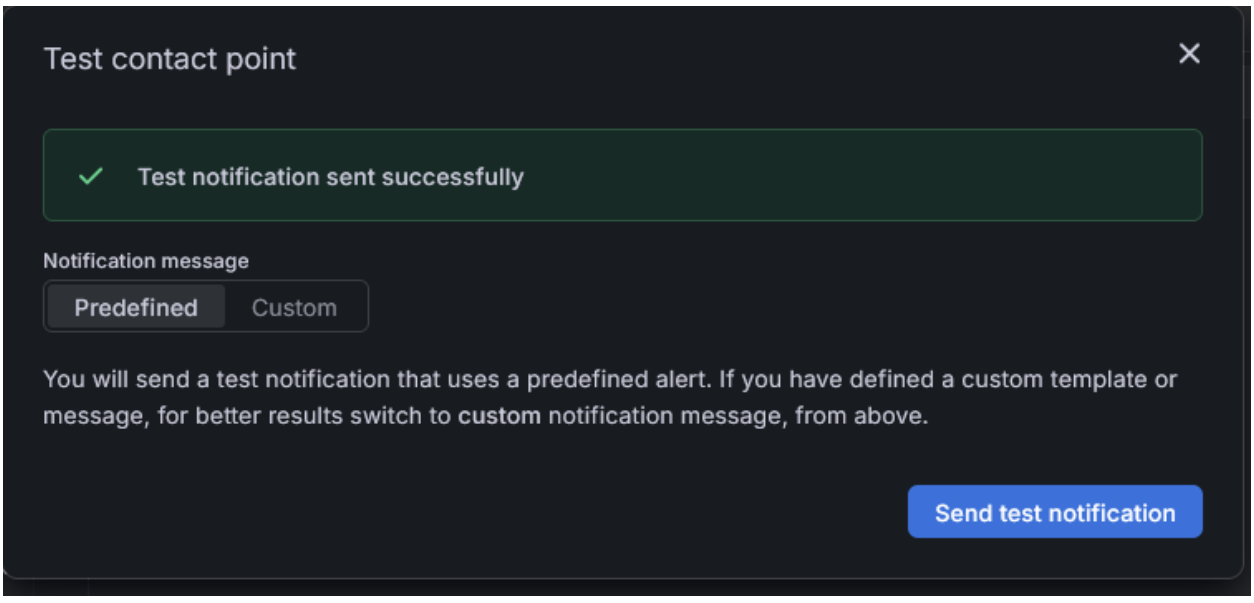
Rellena la configuración. “**Name**” como prefieras, “**Integration**” selecciona “**Telegram**” y **Bot A**PI y **Chat ID** son los datos que hemos conseguido antes en Telegram.



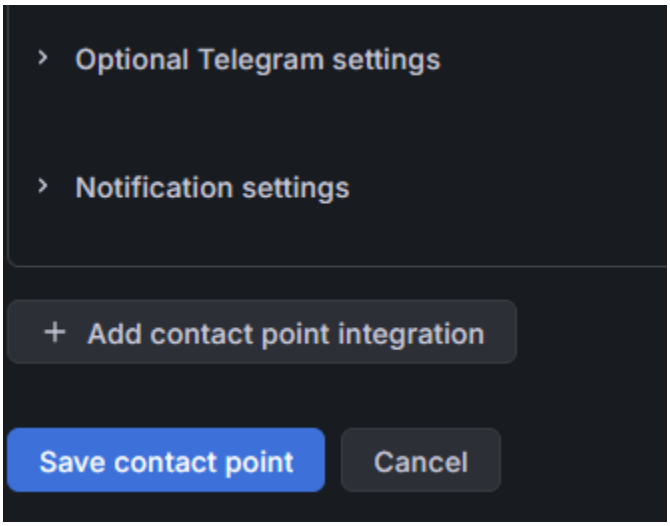
Usa **Test** para probar que funciona.



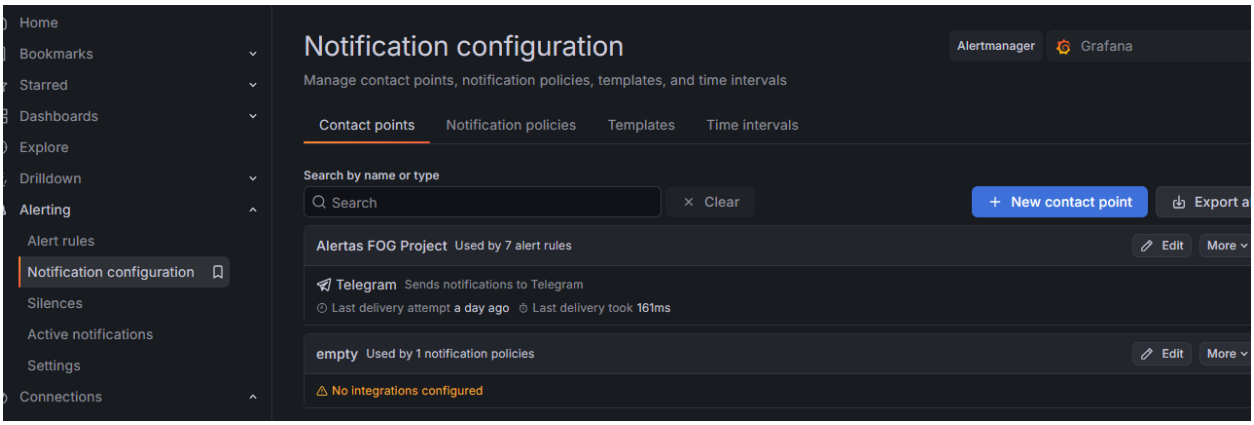

FOG_ALERTS
bot
April 16
Firing
Value: [no value]
Labels:
- alertname = TestAlert
- instance = Grafana
Annotations:
- summary = Notification test
Silence: http://localhost:3000/alerting/silence/new?alertmanager=grafana&matcher=alertname%3DTestAlert&matcher=instance%3DGrafana
19:02
Firing
Value: [no value]
Labels:
- alertname = TestAlert
- instance = Grafana
Annotations:
- summary = Notification test
Silence: http://localhost:3000/alerting/silence/new?alertmanager=grafana&matcher=alertname%3DTestAlert&matcher=instance%3DGrafana
19:02
F
FOG_ALERTS
bot
Message
Mute
Manage
@csa2asir_bot
Username
Add to Group
Bot Privacy Policy
Report
Stop and block bot


Test contact point
Test notification sent successfully
Notification message
Predefined
Custom
You will send a test notification that uses a predefined alert. If you have defined a custom template or message, for better results switch to custom notification message, from above.
Send test notification

Lo guardamos:



Y ahora lo tendremos en **Alerting → Notification configuration**:

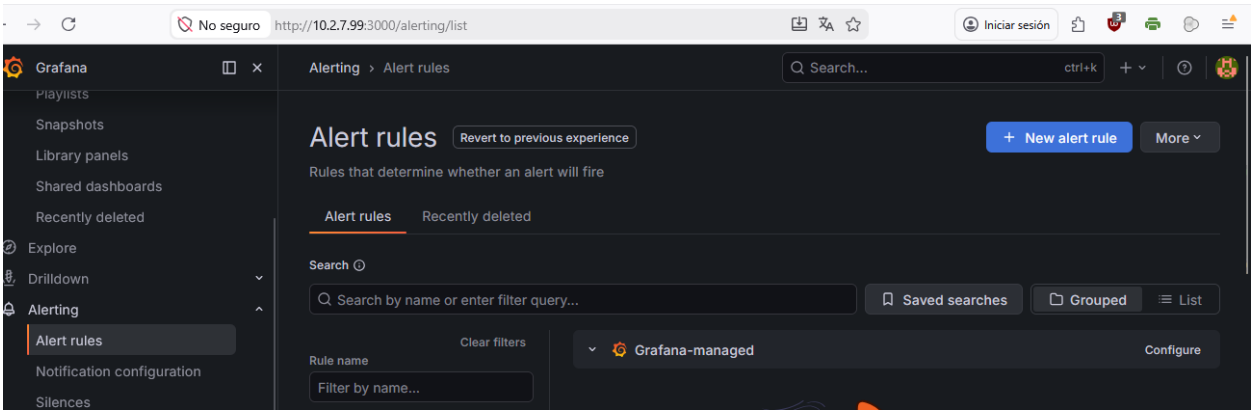


Paso 3: Creación de Alertas

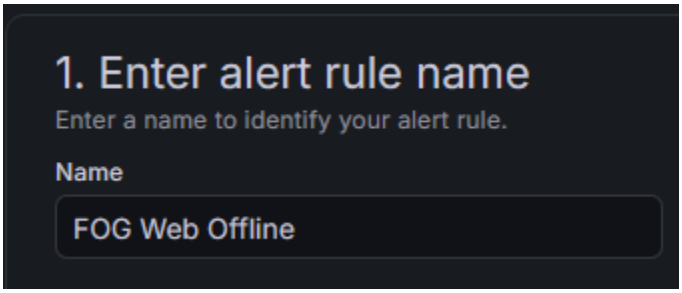
Crear la Alerta de Uptime (Web Caída)

Esta alerta usará los datos del Blackbox Exporter que configuramos antes.

Ve a **Alerting →Alert rules → New alert rule**.



Introduce un nombre:

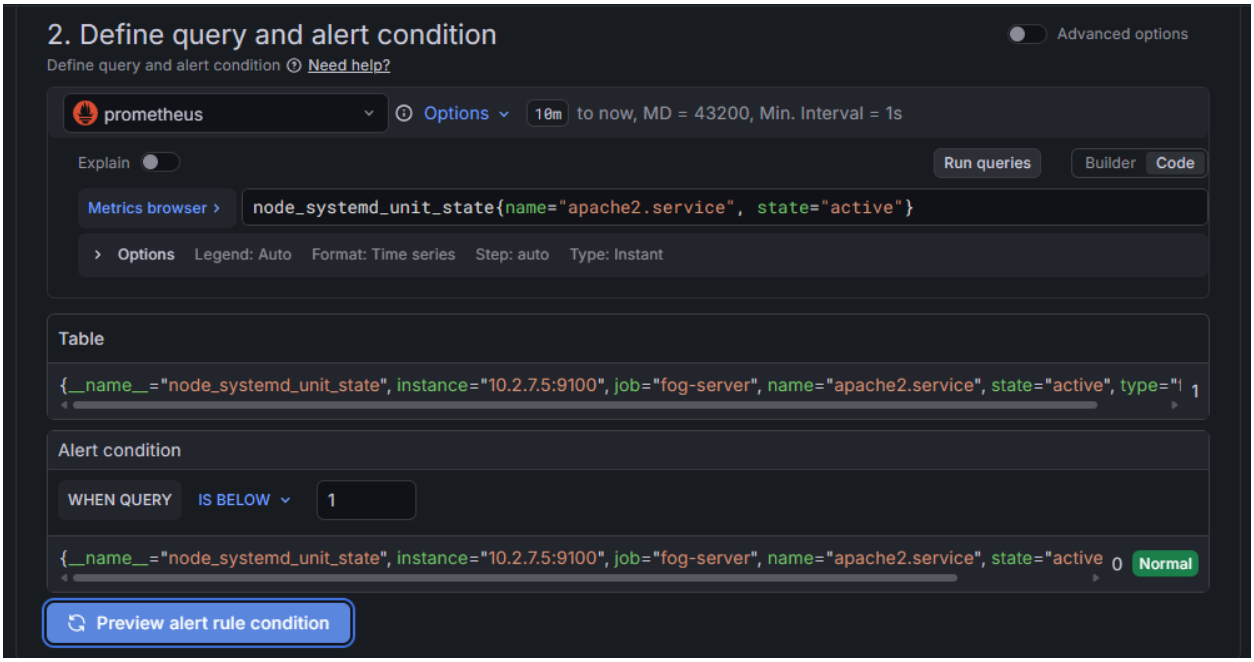


Define:

B. **query** : node_systemd_unit_state{name="apache2.service", state="active"}
C. **Condifción**: “When Query is Below 1”.

Para saber que Query usar, recuerda que podemos ver un listado de métricas que se recogen en el servidor en http://10.2.7.5:9100/metrics, tal y como mencionamos previamente.

Como se puede ver en la imagen de abajo, el estado es “**Normal**” porque el servicio web de FOG está activo.

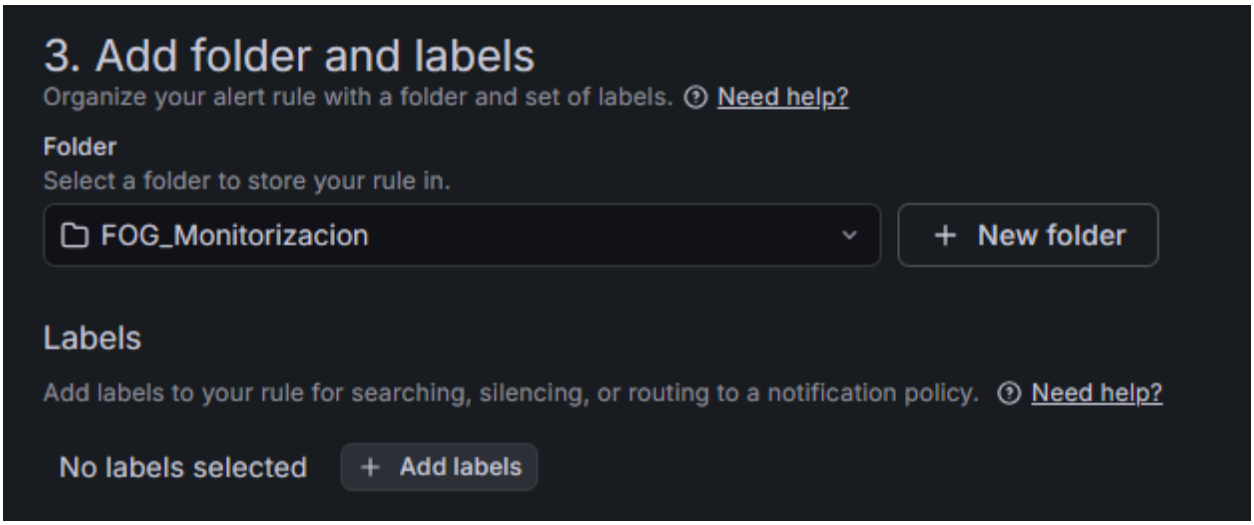


Si paramos el servicio web con sudo systemctl stop apache2

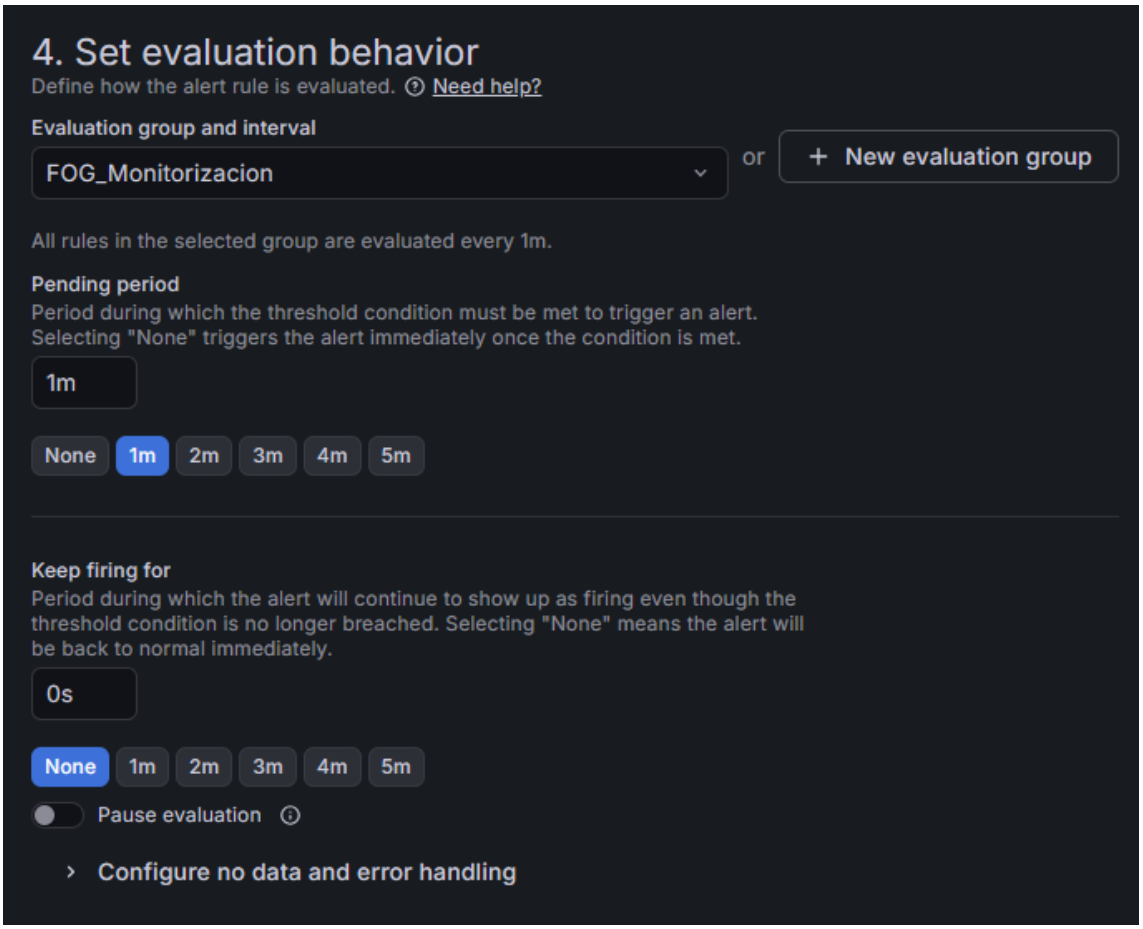
Podremos ver como la alerta se activa.



Añade una carpeta de alertas. En nuestro caso debemos crear una.



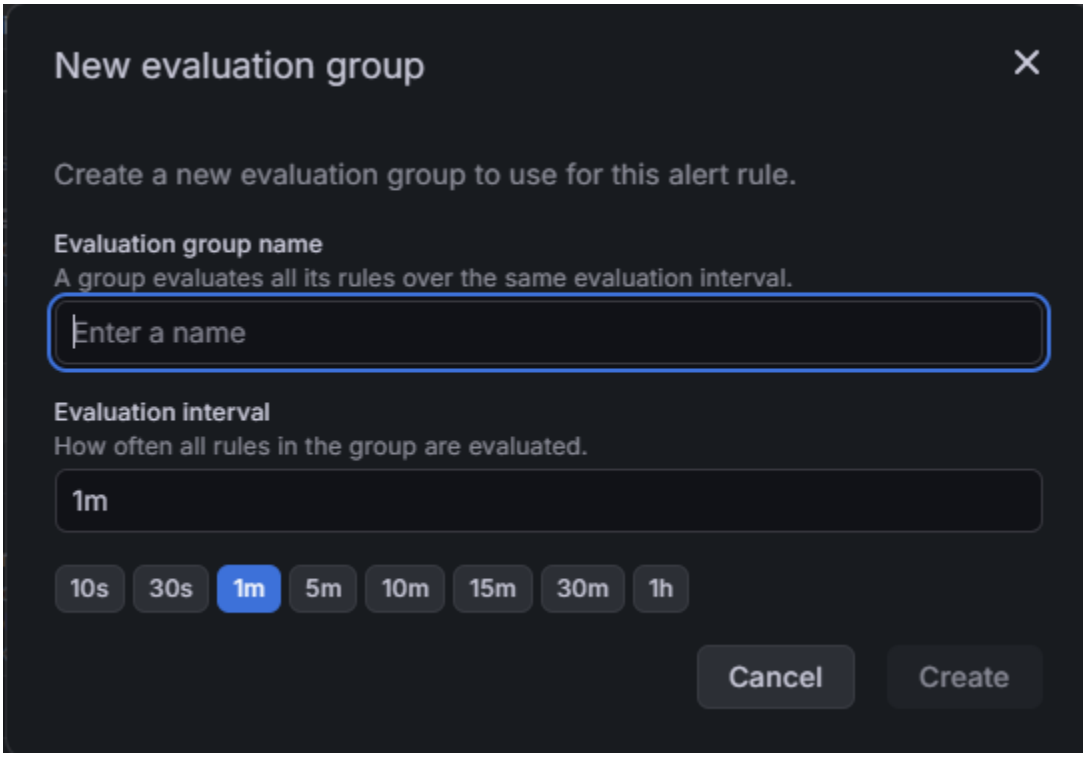
Establece el "**Evaluation Behaviour**".



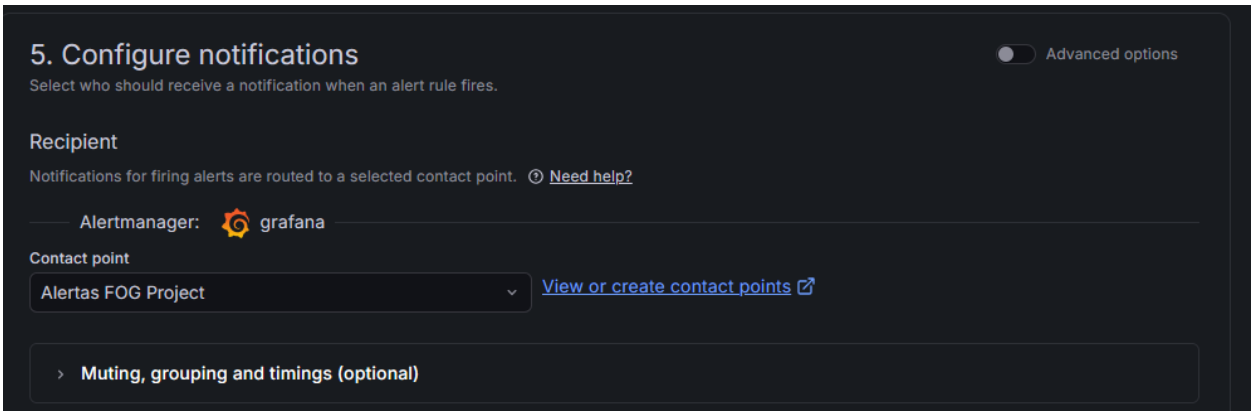
“**Pending Period**” establece cuanto tiempo tiene que pasar para que una alerta esté sucediendo para que ese dispare la alarma.

“**Keep firing for**” establece cada cuanto tiene que sonar la alarma si se sigue cumpliendo la condición.

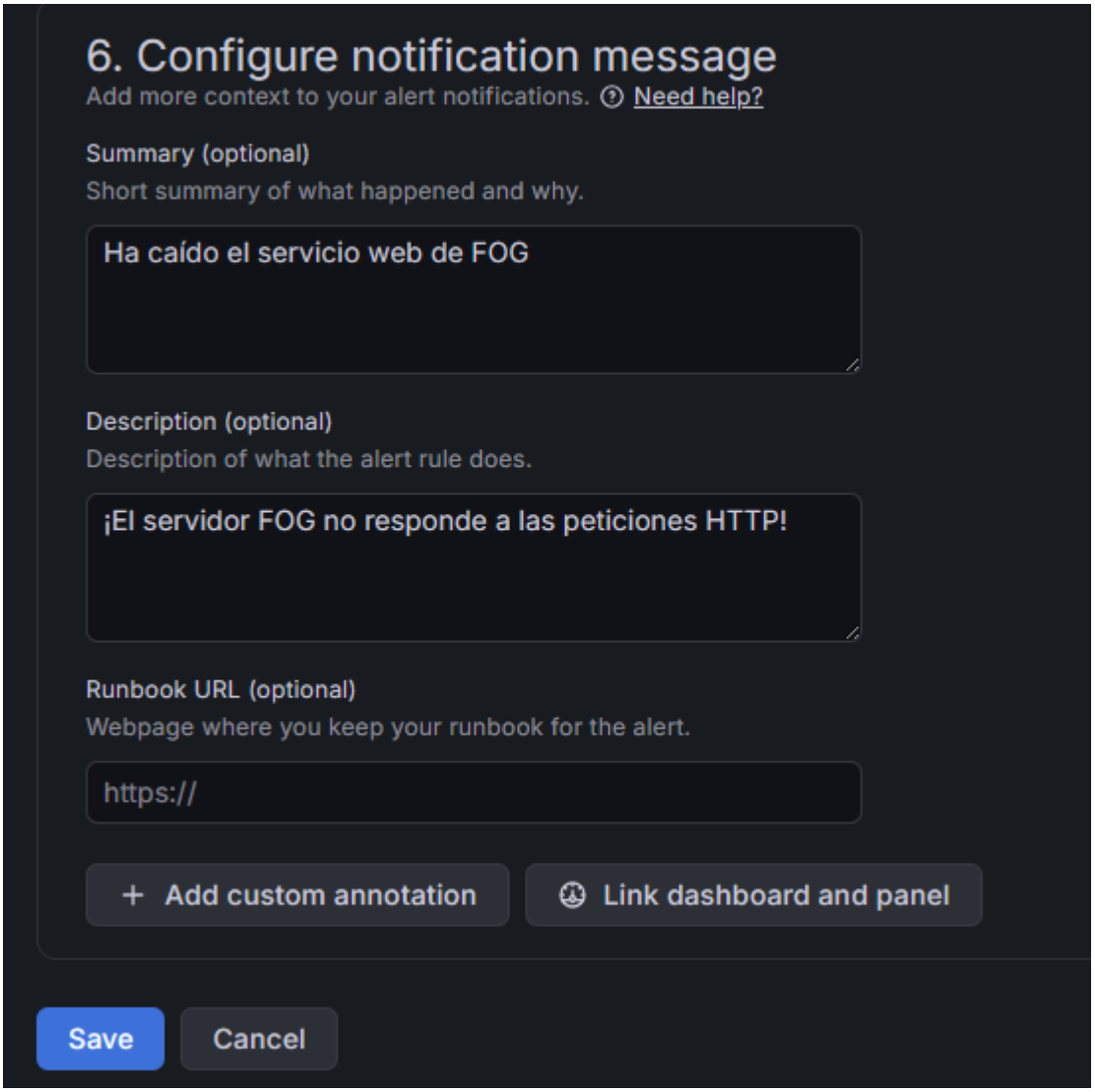
Tendrás que crear un nuevo grupo: Importante establecer el Intervalo de Tiempo por el cual las reglas del grupo serán evaluadas.



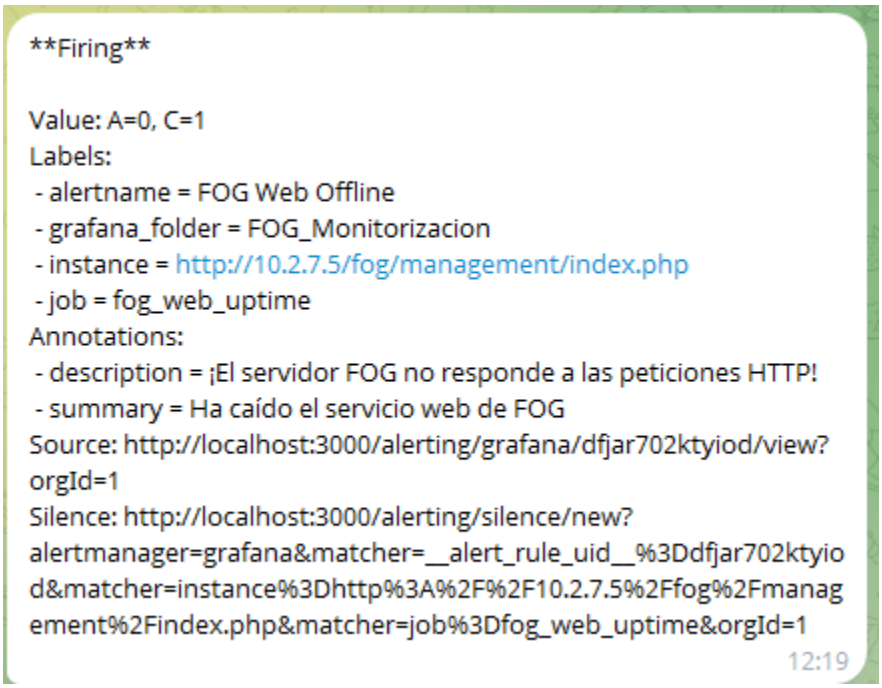
Configuramos las notificaciones usando la que creamos anteriormente para Telegram.



Podemos personalizar el sumario y la descripción del mensaje que nos llegará. Una vez terminado, lo guardamos.



Podemos probar su funcionamiento parando el servicio Web.



Una vez restablecido: Puede tardar unos minutos. Debemos fijarnos en el encabezado de "**Resolved**".



La configuración del resto de alertas que hemos configurado es muy similar, cambiando solamente el Query que hemos utilizado para cada uno:

D. **mariadb.service**: La base de datos.

```
node_systemd_unit_state{name="man-db.timer",state="inactive",type=""} 0
node_systemd_unit_state{name="mariadb.service",state="activating",type="notify"} 0
node_systemd_unit_state{name="mariadb.service",state="active",type="notify"} 1
node_systemd_unit_state{name="mariadb.service",state="deactivating",type="notify"} 0
node_systemd_unit_state{name="mariadb.service",state="failed",type="notify"} 0
node_systemd_unit_state{name="mariadb.service",state="inactive",type="notify"} 0
node_systemd_unit_state{name="modprobe@configfs.service",state="activating",type="ones
```

Query: `node_systemd_unit_state{name="mariadb.service", state="active"}`

Condición: **When Query is Below 1.**

E. **FOGMulticastManager.service**: Gestión de despliegues masivos a varios PCs a la vez.

```
node_systemd_unit_state{name="FOGMulticastManager.service",state="activating",type="simple"} 0
node_systemd_unit_state{name="FOGMulticastManager.service",state="active",type="simple"} 1
node_systemd_unit_state{name="FOGMulticastManager.service",state="deactivating",type="simple"} 0
node_systemd_unit_state{name="FOGMulticastManager.service",state="failed",type="simple"} 0
node_systemd_unit_state{name="FOGMulticastManager.service",state="inactive",type="simple"} 0
node_systemd_unit_state{name="FOGPingHosts.service",state="activating",type="simple"} 0
```

Query:

```
node_systemd_unit_state{name="FOGMulticastManager.service", state="active"}
```

Condición: **When Query is Below 1.**

Summary: “El servicio de despliegue masivo en {{ $labels.instance }} no está funcionando”.

F. **FOGScheduler.service**: Ejecución de tareas programadas (inventarios, limpiezas).

```
node_systemd_unit_state{name="FOGScheduler.service",state="activating",type="simple"} 0
node_systemd_unit_state{name="FOGScheduler.service",state="active",type="simple"} 1
node_systemd_unit_state{name="FOGScheduler.service",state="deactivating",type="simple"} 0
node_systemd_unit_state{name="FOGScheduler.service",state="failed",type="simple"} 0
node_systemd_unit_state{name="FOGScheduler.service",state="inactive",type="simple"} 0
```

Query: `node_systemd_unit_state{name="FOGScheduler.service", state="active"}`

Condición: **When Query is Below 1.**

Summary: FOG Scheduler está CAÍDO

Description: El servicio encargado de las tareas programadas en el servidor FOG ({{ $labels.instance }}) no responde.

G. **"FOG Server: Disco Casi Lleno"**[3]: Nos indicará cuando el almacenamiento del servidor llega al 90% de su capacidad.

```
node_filefd_maximum 9.223372036854776e+18
# HELP node_filesystem_avail_bytes Filesystem space available to non-root users in bytes.
# TYPE node_filesystem_avail_bytes gauge
node_filesystem_avail_bytes{device="/dev/mapper/ubuntu--vg-ubuntu--lv",fstype="ext4",mountpoint="/"} 6.7047018496e+10
node_filesystem_avail_bytes{device="/dev/sda2",fstype="ext4",mountpoint="/boot"} 1.707094016e+09
# HELP node_filesystem_device_error Whether an error occurred while getting statistics for the given device.
```

Query:

```
(1 - node_filesystem_avail_bytes{mountpoint="/"} /
node_filesystem_size_bytes{mountpoint="/"}) * 100
```

Condición: When query is above 100

Summary: ¡Peligro! El uso de disco en el servidor FOG supera el 90%. Espacio actual: {{ printf "%.2f" $values.A.Value }}%

[3] https://community.grafana.com/t/set-alerts-for-when-cpu-load-hard-disk-reaches-85-in-grafana-datasource-using-prometheus/70800

Paso 4: Creación de alerta para el uso de CPU del servidor

Query:

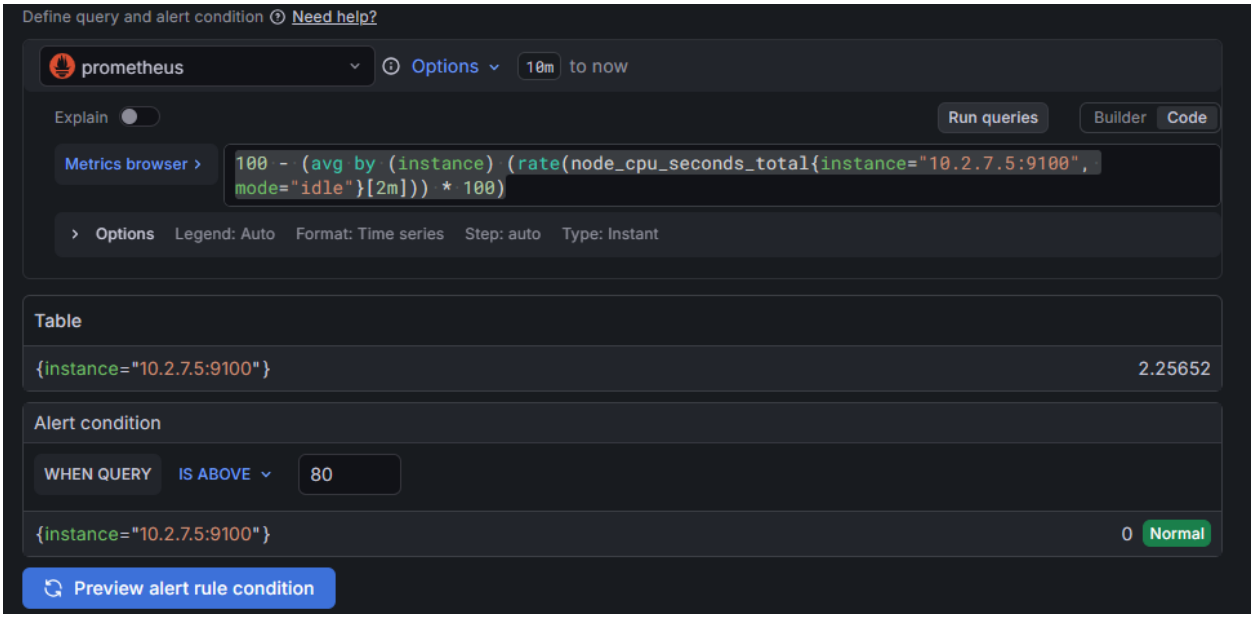
```
100 - (avg by (instance) (rate(node_cpu_seconds_total{instance="10.2.7.5:9100", mode="idle"}[2m])) * 100)
```

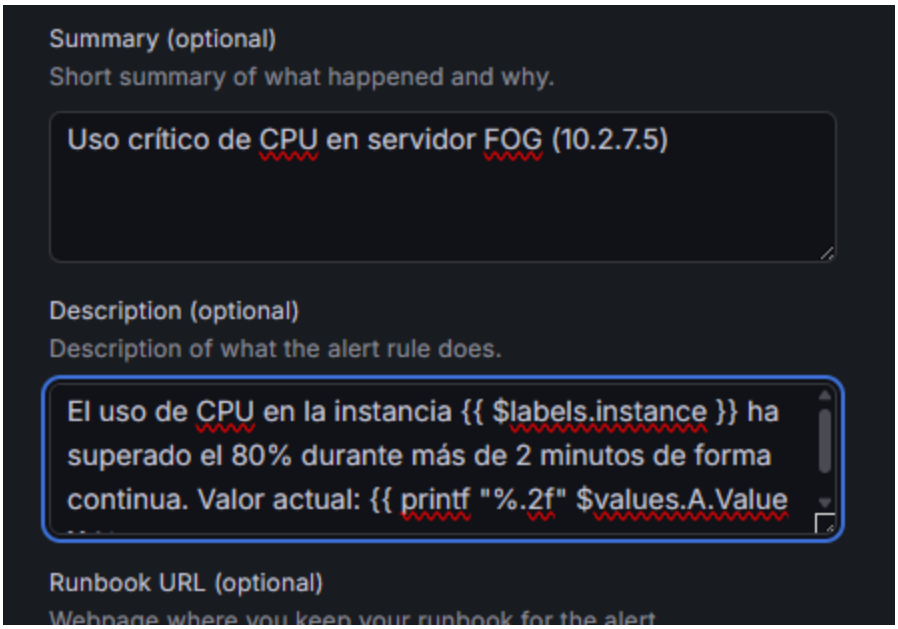
Condición: When query is above 80

Vamos a probarlo poníendolo a 5.

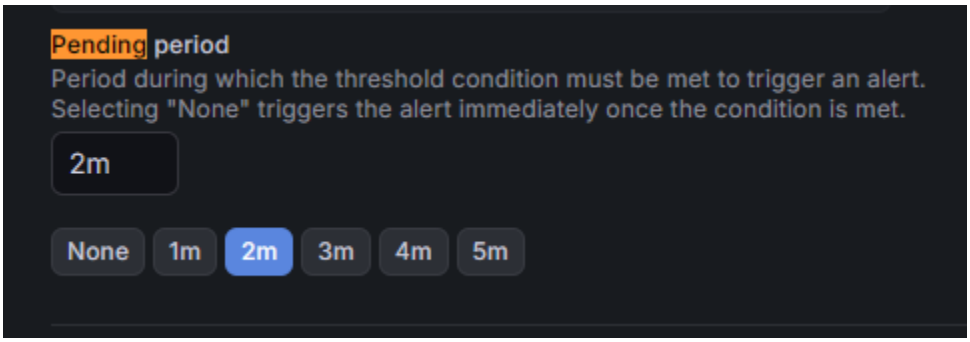
Summary: Uso crítico de CPU en servidor FOG (10.2.7.5)

Description: El uso de CPU en la instancia {{ $labels.instance }} ha superado el 80% durante más de 2 minutos de forma continua. Valor actual: {{ printf "%.2f" $values.A.Value }}%

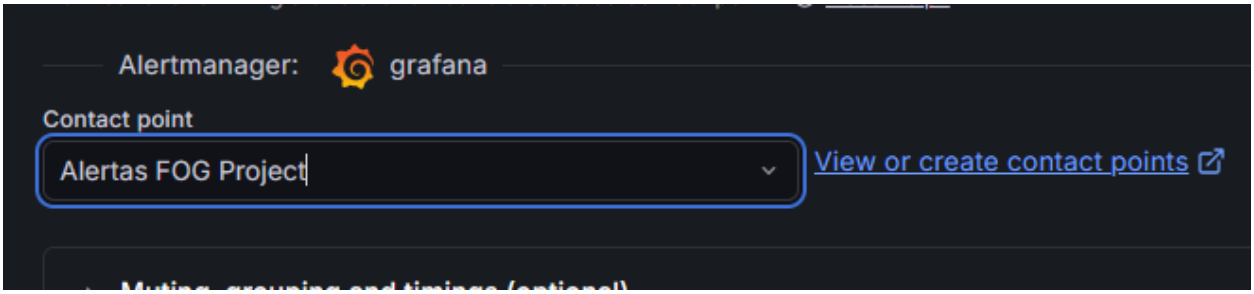




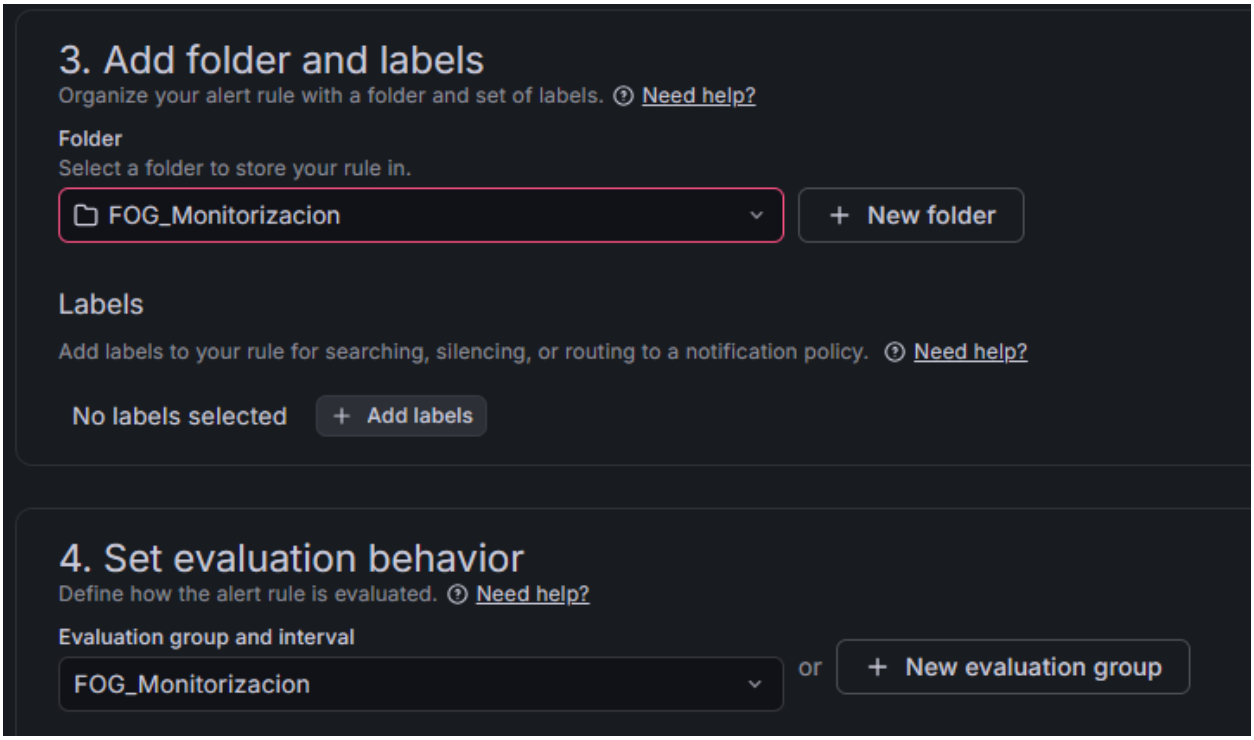
Pending period: 2 minutos



Establece el Contact Point: El que hemos creado previamente



También debes seleccionar una carpeta y un grupo de evaluación. Ambos han sido creados en pasos anteriores.



Prueba de la regla:

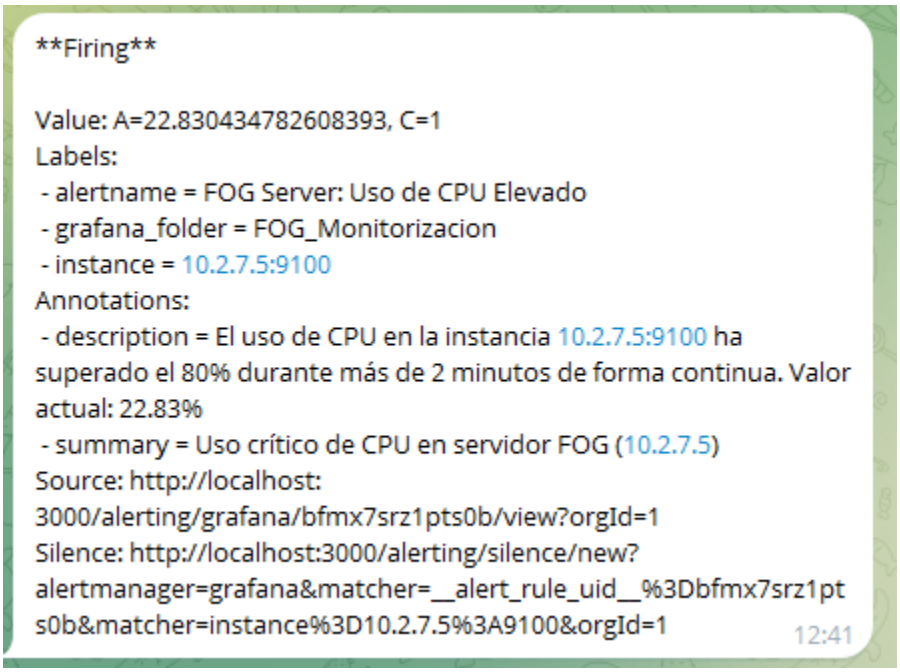
Hemos usado el comando para crear un bucle infinito

```
timeout 3m sha1sum /dev/zero
```

Y hemos bajado el umbral hasta 10. En el Dashboard vemos como va subiendo la CPU

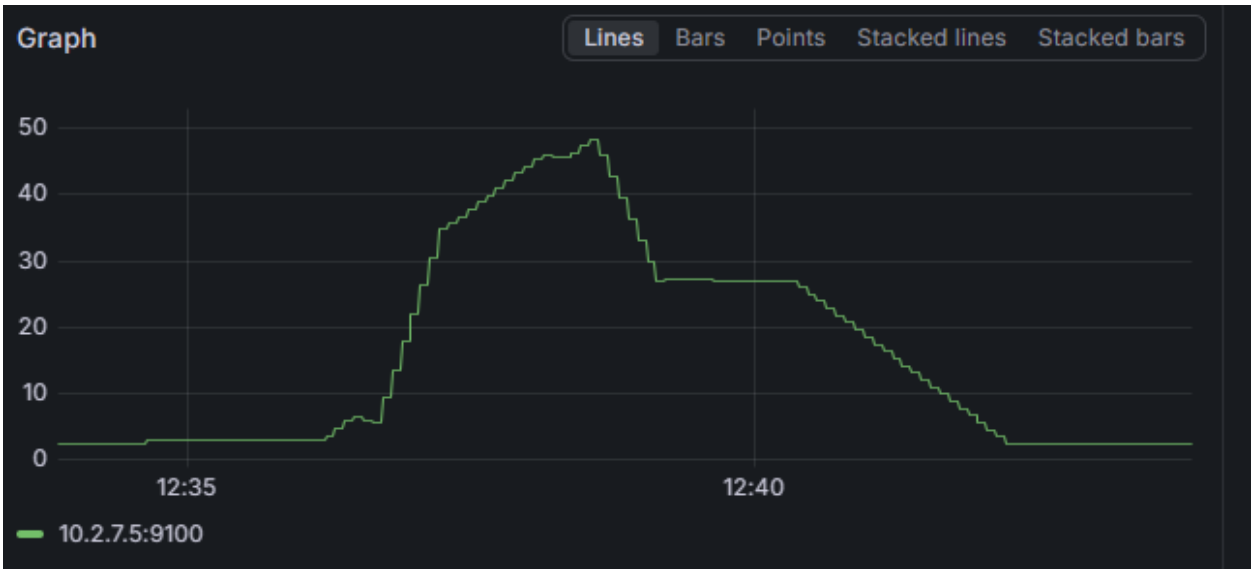


A los dos minutos salta la alarma:



En el mensaje, el 80% es texto, no es un valor como la parte de “Valor actual”. Hay que tener en cuenta que hemos bajado el umbral hasta el 5%.

Progreso de la CPU durante la prueba.



Una vez que hemos parado el comando, al poco tiempo se “resuelve”.

